第8章 图

提纲

CONTENTS

8.1 图的基本概念

8.2 图的存储结构

8.3 图的遍历

8.3 图遍历算法的应用

8.5 生成树和最小生成树

8.6 最短路径

8.7 拓扑排序

8.8 AOE网与关键路径

8.1 图的基本概念

8.1.1 图的定义

- 图G（Graph）由两个集合V（Vertex）和E（Edge）组成，记为G=(V，E)。
- V是顶点的有限集合，记为V(G)。
- E是连接V中两个不同顶点（顶点对）的边的有限集合，记为E(G)。

抽象数据类型图的描述

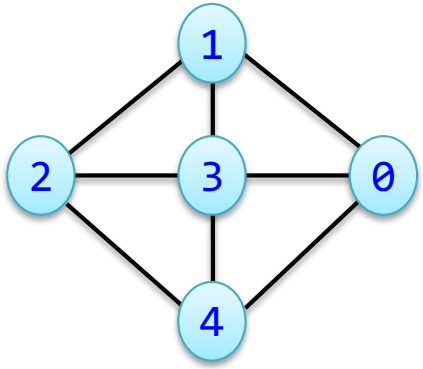
```
ADT Graph
{
数据对象:
    D={a_i | 0≤i≤n-1, n≥0, a_i为int类型}      //a_i为每个顶点的唯一编号
数据关系:
    R={r}
    r={<a_i, a_j> | a_i, a_j∈D, 0≤i≤n-1, 0≤j≤n-1, 其中a_i可以有零个
          或多个前驱元素，可以有零个或多个后继元素 }
基本运算:
    void CreateGraph(): 根据相关数据建立一个图。
    void DispGraph(): 输出一个图。
     …
}
```

说明 约定用i（$0 \leqslant i \leqslant n-1$）表示第$i$个顶点的编号。

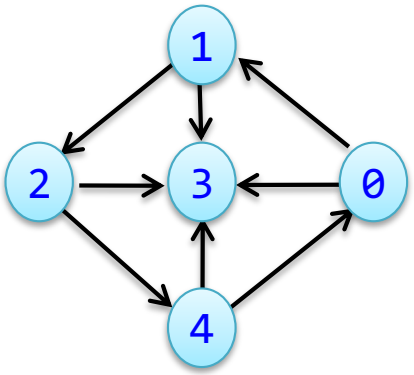
无向图和有向图

- 在图G中，如果代表边的顶点对（或序偶）是无序的，则称G为无向图。无向图中代表边的无序顶点对通常用圆括号括起来，用以表示一条无向边。

- 如果表示边的顶点对（或序偶）是有序的，则称G为有向图。在有向图中代表边的顶点对通常用尖括号括起来，用以表示一条有向边（又称为弧），如<i，j>表示从顶点i到顶点j的一条边。



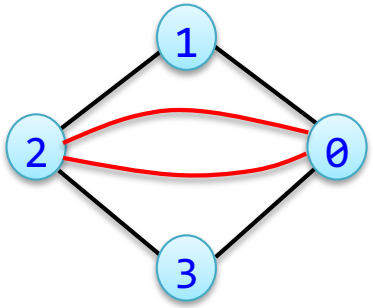
（a）一个无向图



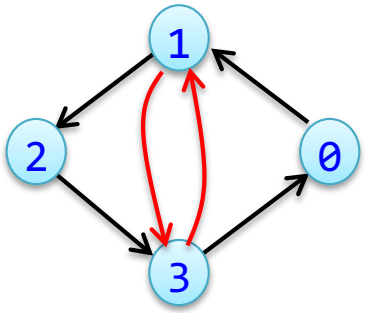
（b）一个有向图



数据结构中的图一般不重复出现一条边，如果允许重复边出现，这样的图称为多重图，如一个无向图中顶点1和2之间出现两条或两条以上的边。本书中讨论的图均指非多重图。



（a）多重无向图

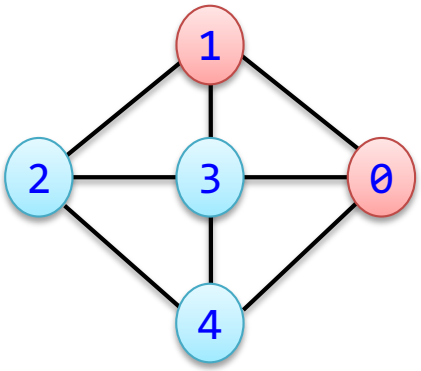


（b）多重有向图

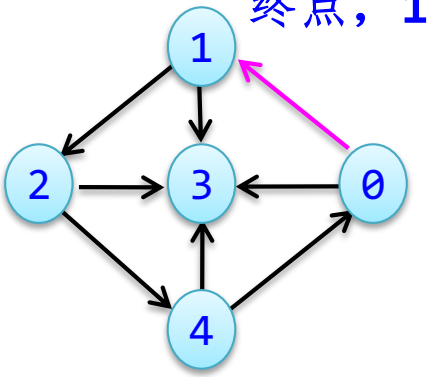
8.1.2 图的基本术语

- 在一个无向图中，若存在一条边(*i*，*j*)，则称顶点*i*和顶点*j*为该边的两个端点，并称它们互为邻接点，即顶点*i*是顶点*j*的一个邻接点，顶点*j*也是顶点*i*的一个邻接点。

- 在一个有向图中，若存在一条边<*i*，*j*>，则称此边是顶点*i*的一条出边，同时也是顶点*j*的一条入边。*i*和*j*分别为此边的起始端点（简称为起点）和终止端点（简称终点）。并称顶点*j*是*i*的出边邻接点，顶点*i*是*j*的入边邻接点。



（a）一个无向图



终点，1是顶点0的出边邻接点

起点，0是顶点1的入边邻接点

（b）一个有向图

- 在无向图中，顶点所关联的边的数目称为该顶点的度。在有向图中，顶点i的度又分为入度和出度，以顶点i为终点的入边的数目，称为该顶点的入度。以顶点i为起点的出边的数目，称为该顶点的出度。一个顶点的入度与出度的和为该顶点的度。

- 若一个图（无论有向图或无向图）中有n个顶点和e条边，每个顶点的度为d_i（$0 \leqslant i \leqslant n-1$），则有：

$$e = \frac{1}{2}\sum_{i=0}^{n-1} d_i$$

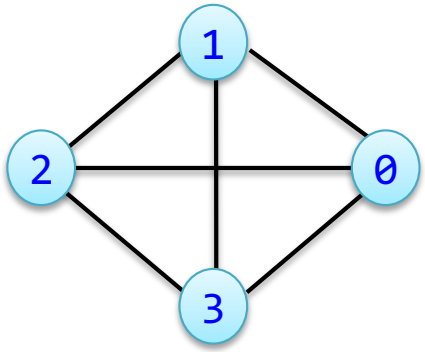
【例】一个无向图中有16条边，度为4的顶点有3个，度为3的顶点有4个，其余顶点的度均小于3，则该图至少有多少个顶点？



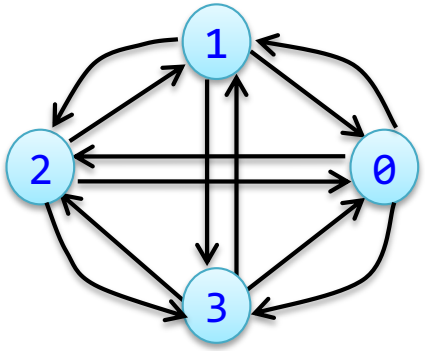
设该图有n个顶点，图中度为i的顶点数为n_i（$0 \leqslant i \leqslant 4$）。

- $n_4=3$，$n_3=4$。
- 要使顶点数最少，该图应是连通的，即$n_0=0$。
- $n=n_4+n_3+n_2+n_1+n_0=7+n_2+n_1$，即$n_2+n_1=n-7$。
- 度之和$=4\times3+3\times4+2\times n_2+n_1=24+2n_2+n_1 \leqslant 24+2(n_2+n_1)=$
  $24+2\times(n-7)=10+2n$。
- 而度之和$=2e=32$，所以有$10+2n \geqslant 32$，即$n \geqslant 11$。
- 即这样的无向图至少有11个顶点。

- 完全无向图中的每两个顶点之间都存在着一条边。含有***n***个顶点的完全无向图有***n*(*n*-1)/2**条边。
- 完全有向图中的每两个顶点之间都存在着方向相反的两条边。含有***n***个顶点的完全有向图包含有***n*(*n*-1)**条边。



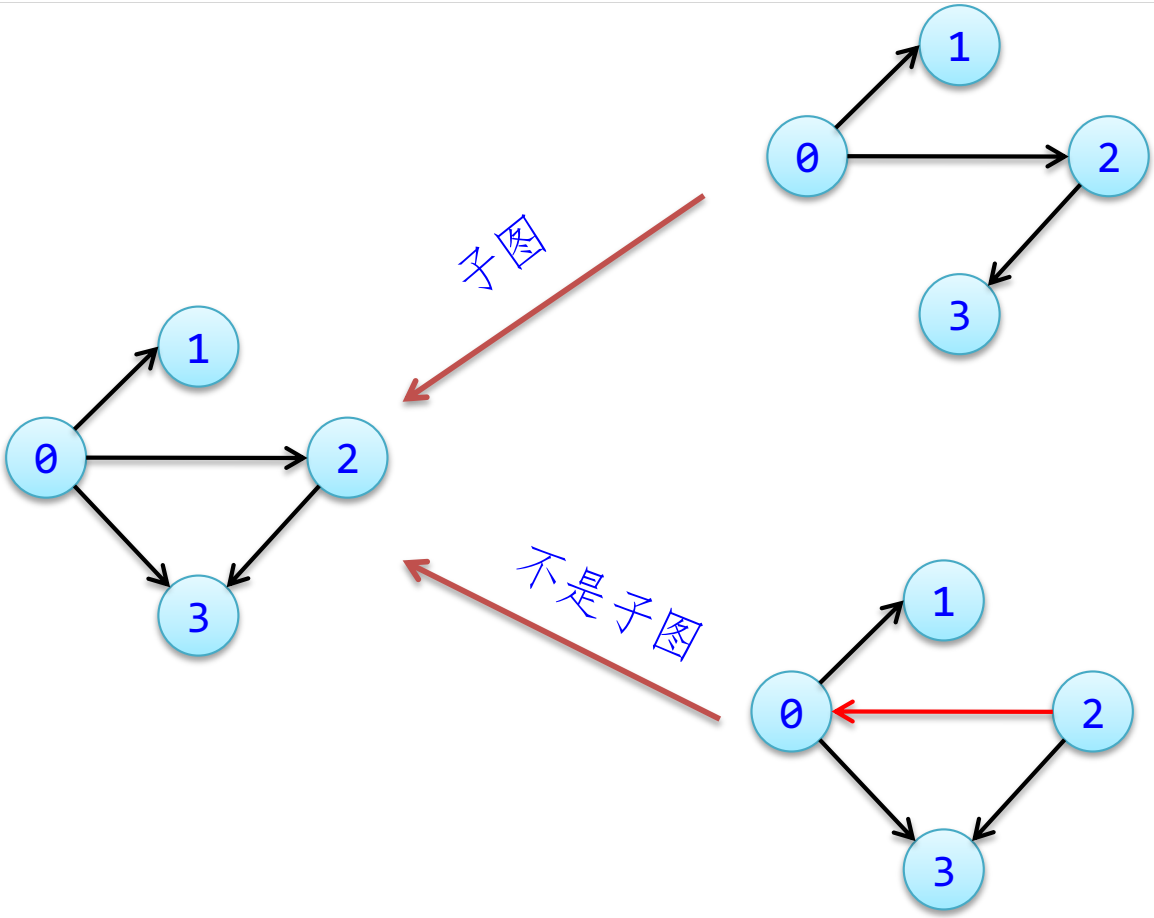
（a）一个完全无向图



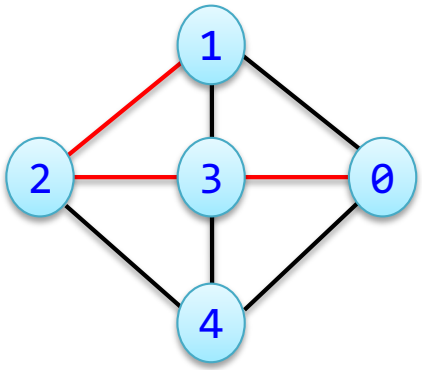
（b）一个完全有向图

- 当一个图接近完全图时，则称为稠密图。
- 当一个图含有较少的边数（即无向图有$e<<n(n-1)/2$，有向图有$e<<n(n-1)$）时，则称为稀疏图。

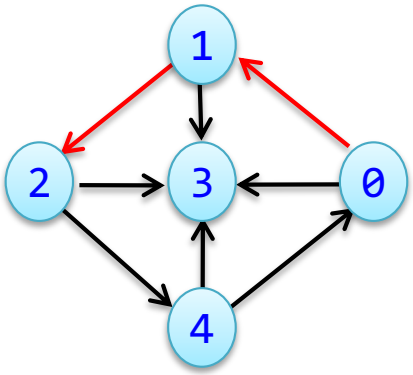
- 设有两个图G=(V，E)和G'=(V'，E')，若V'是V的子集，即V'⊆V，且E'是E的子集，即E'⊆E，则称G'是G的子图。



- 在一个图G=(V，E)中，从顶点i到顶点j的一条路径是一个顶点序列$(i, i_1, i_2, \cdots, i_m, j)$，若此图G是无向图，则边$(i, i_1)$，$(i_1, i_2)$，…，$(i_{m-1}, i_m)$，$(i_m, j)$属于E(G)；若此图是有向图，则$<i, i_1>$，$<i_1, i_2>$，…，$<i_{m-1}, i_m>$，$<i_m, j>$属于E(G)。
- 路径长度是指一条路径上经过的边的数目。
- 若一条路径上除开始点和结束点可以相同外，其余顶点均不相同，则称此路径为简单路径。

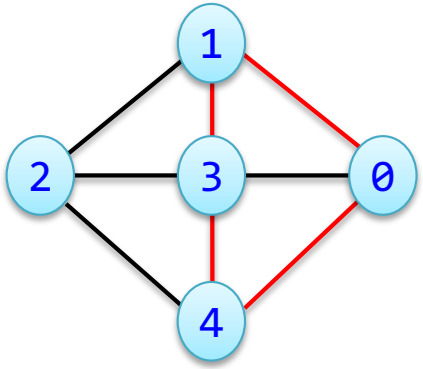


（0,3,2,1）的简单路径长度为3

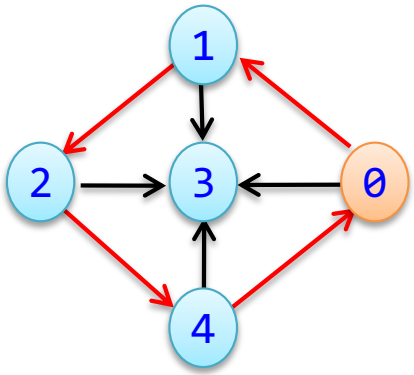


（0,1,2）的简单路径长度为2

- 若一条路径上的开始点与结束点为同一个顶点，则此路径被称为回路或环。
- 开始点与结束点相同的简单路径被称为简单回路或简单环。

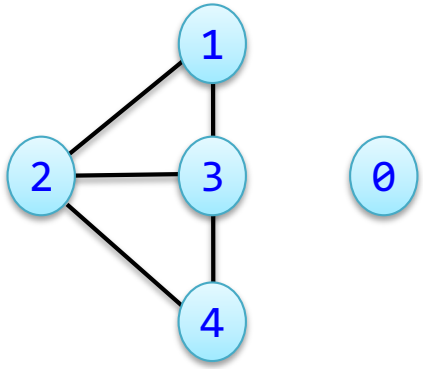


（0,1,3,4,0）的
简单回路长度为4



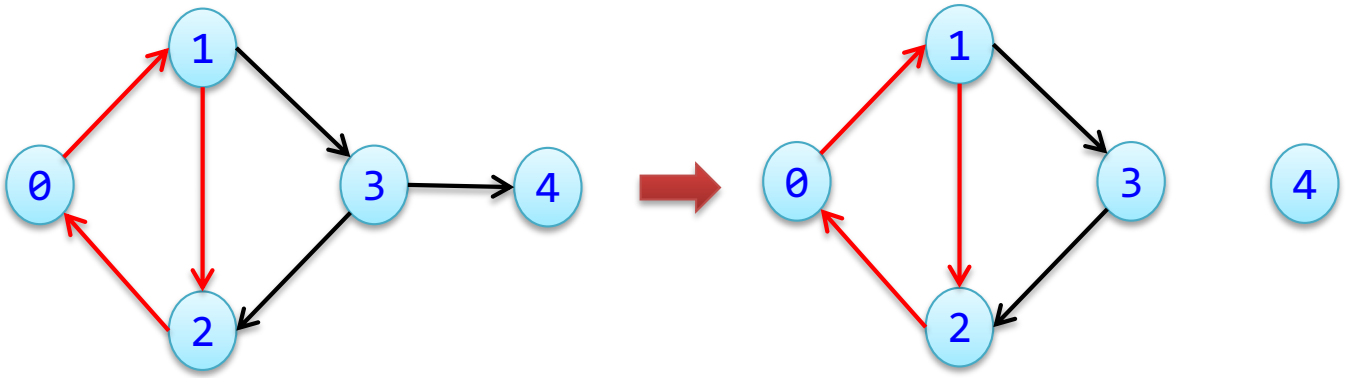
（0,1,2,4,0）的
简单回路长度为4

- 在无向图G中，若从顶点i到顶点j有路径，则称顶点i和顶点j是连通的。
- 若图G中任意两个顶点都连通，则称G为连通图，否则称为非连通图。
- 无向图G中的极大连通子图称为G的连通分量。显然，任何连通图的连通分量只有一个即本身，而非连通图有多个连通分量。

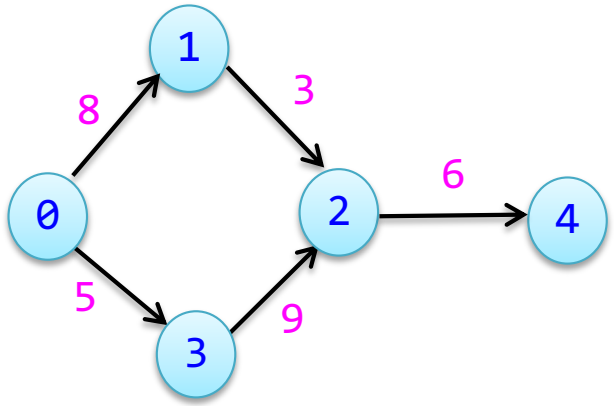


两个连通分量构成

- 在有向图G中，若从顶点i到顶点j有路径，则称从顶点i到顶点j是连通的。
- 若图G中的任意两个顶点i和j都连通，即从顶点i到顶点j和从顶点j到顶点i都存在路径，则称图G是强连通图。
- 有向图G中的极大强连通子图称为G的强连通分量。
- 显然，强连通图只有一个强连通分量即本身，非强连通图有多个强连通分量。一般地单个顶点自身就是一个强连通分量。



- 图中每一条边都可以附有一个对应的数值，这种与边相关的数值称为权。权可以表示从一个顶点到另一个顶点的距离或花费的代价。
- 边上带有权的图称为带权图，也称作网。



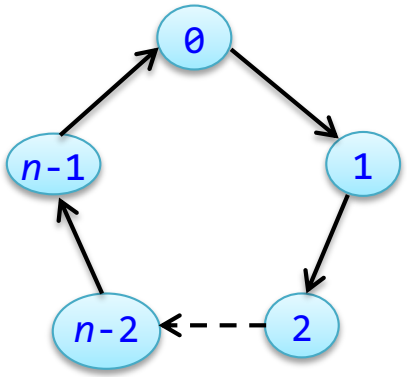
【例8.1】n个顶点的强连通图至少有多少条边？这样的有向图是什么形状？



根据强连通图的定义可知，图中的任意两个顶点i和j都连通，即从顶点i到顶点j和从顶点j到顶点i都存在路径。这样，每个顶点的度$d_i \geq 2$，设图中总的边数为e，有：

$$e = \frac{1}{2}\sum_{i=0}^{n-1} d_i \geq \frac{1}{2}\sum_{i=0}^{n-1} 2 = n$$

即$e \geq n$。因此，n个顶点的强连通图至少有n条边，刚好只有n条边的强连通图是环形的，即顶点0到顶点1有一条有向边，顶点1到顶点2有一条有向边，…，顶点n-1到顶点0有一条有向边。



8.2 图的存储结构

8.2.1 邻接矩阵

1. 邻接矩阵存储方法

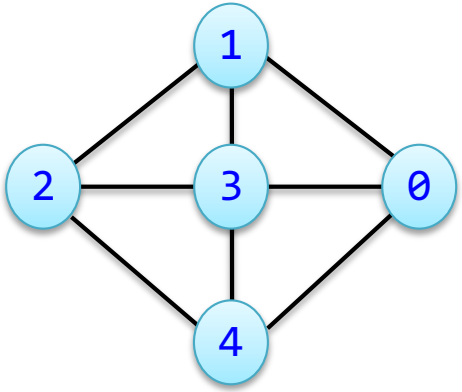
邻接矩阵是表示顶点之间邻接关系的矩阵。设G=(V，E)是含有*n*（设*n*>0）个顶点的图，各顶点的编号为0～*n*-1，则G的邻接矩阵数组A是*n*阶方阵。

（1）如果G是不带权图，则：

$$A[i][j]=\begin{cases}1 & \text{若}(i,j)\in E(G)\text{或者}\langle i,j\rangle\in E(G)\\ 0 & \text{其他}\end{cases}$$

（2）如果G是带权图，则：

$$A[i][j]=\begin{cases}w_{ij} & \text{若}i\neq j\text{并且}(i,j)\in E(G)\text{或者}\langle i,j\rangle\in E(G)\\ 0 & \text{若}i=j\\ \infty & \text{其他}\end{cases}$$

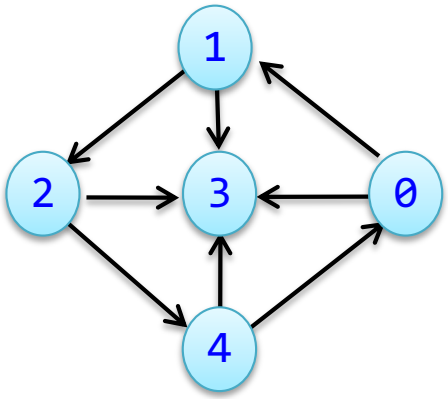


（a）一个无向图

$$A_1 = \begin{bmatrix} 0 & 1 & 0 & 1 & 1 \\ 1 & 0 & 1 & 1 & 0 \\ 0 & 1 & 0 & 1 & 1 \\ 1 & 1 & 1 & 0 & 1 \\ 1 & 0 & 1 & 1 & 0 \end{bmatrix}$$

对称

说明 **无向图的邻接矩阵一定对称！**



（b）一个有向图

$$A_2 = \begin{bmatrix} 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 \end{bmatrix}$$

不对称

说明 **有向图的邻接矩阵不一定对称！**

邻接矩阵的特点

- 图的邻接矩阵表示是唯一的。
- 对于含有n个顶点的图，采用邻接矩阵存储时，无论是有向图还是无向图，也无论边的数目是多少，其存储空间均为$O(n^2)$，所以邻接矩阵适合于存储边数较多的稠密图。
- 无向图的邻接矩阵一定是一个对称矩阵。因此在顶点个数n很大时可以采用对称矩阵的压缩存储方法减少存储空间。
- 对于无向图，邻接矩阵的第i行（或第i列）非零元素（或非∞元素）的个数正好是顶点i的度。
- 对于有向图，邻接矩阵的第i行（或第i列）非零元素（或非∞元素）的个数正好是顶点i的出度（或入度）。
- 用邻接矩阵方法存储图，确定任意两个顶点之间是否有边相连的时间为$O(1)$。

图的邻接矩阵类MatGraph

```
const int MAXV=100;                    //图中最多的顶点数
const int INF=0x3f3f3f3f;              //用INF表示∞
class MatGraph                         //图邻接矩阵类
{
public:
    int edges[MAXV][MAXV];             //邻接矩阵数组，假设元素为int类型
    int n,e;                           //顶点数，边数
    string vexs[MAXV];                 //存放顶点信息
    //图的基本运算算法
}
```

邻接矩阵数组

2．图基本运算在邻接矩阵中的实现

（1）创建图的邻接矩阵

- 邻接矩阵数组*a*
- 顶点数*n*
- 边数*e*



邻接矩阵g

```
void CreateMatGraph(int a[][MAXV],int n,int e)
//通过a、n和e来建立图的邻接矩阵
{  this->n=n; this->e=e;                    //置顶点数和边数
   for (int i=0;i<n;i++)
      for (int j=0;j<n;j++)
          this->edges[i][j]=a[i][j];
}
```

（2）输出图

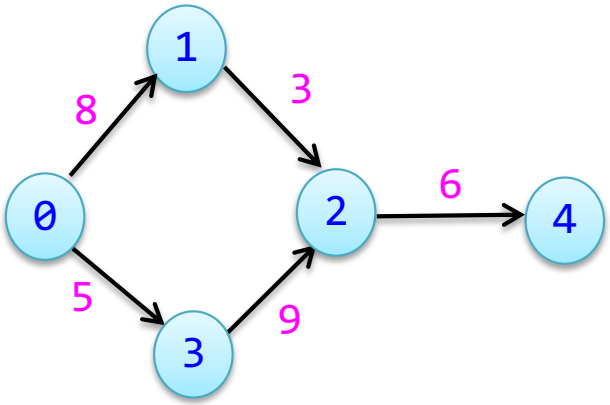
```
void DispMatGraph()              //输出图的邻接矩阵
{  for (int i=0;i<n;i++)
   {  for (int j=0;j<n;j++)
         if (edges[i][j]==INF)
            printf("%4s","∞");
         else
            printf("%4d",edges[i][j]);
      printf("\n");
   }
}
```



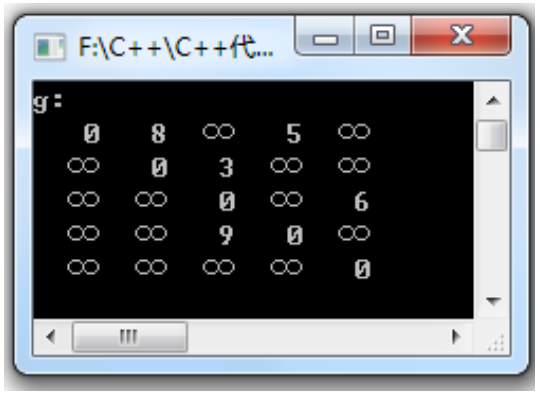
将图的邻接矩阵类型定义及其基本运算存放在MatGraph.cpp文件中

程序验证

```
int main()
{   MatGraph g;
    int n=5,e=5;
    int a[MAXV][MAXV]={{0,8,INF,5,INF},{INF,0,3,INF,INF},
                       {INF,INF,0,INF,6},{INF,INF,9,0,INF},
                        {INF,INF,INF,INF,0}};
    g.CreateMatGraph(a,n,e);
    printf("g:\n");
    g.DispMatGraph();
    return 0;
}
```



一个带权有向图



【例8.2】一个含有n个顶点e条边的图采用邻接矩阵g存储，设计以下算法：

（1）该图为无向图，求其中顶点v的度。

（2）该图为有向图，求该图中顶点v的出度和入度。



- 无向图，求其中顶点v的度

```
#include"MatGraph.cpp"              //包含图(邻接矩阵)的基本运算算法
int Degree1(MatGraph& g,int v)     //无向图邻接矩阵g中求顶点v的度
{  int d=0;
   for (int j=0;j<g.n;j++)         //统计第v行的非0非∞元素个数
      if (g.edges[v][j]!=0 && g.edges[v][j]!=INF)
         d++;
   return d;
}
```

- 有向图，求该图中顶点v的出度和入度

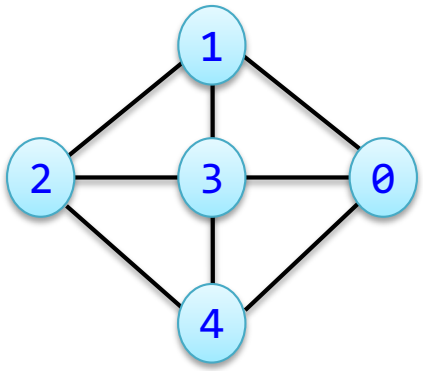
```
vector<int> Degree2(MatGraph& g,int v)
//有向图邻接矩阵g中求顶点v的出度和入度
{  vector<int> ans={0,0};              //ans[0]累计出度,ans[1]累计入度
   for (int j=0;j<g.n;j++)             //统计第v行的非0非∞元素个数为出度
       if (g.edges[v][j]!=0 && g.edges[v][j]!=INF)
          ans[0]++;
   for (int i=0;i<g.n;i++)             //统计第v列的非0非∞元素个数为入度
       if (g.edges[i][v]!=0 && g.edges[i][v]!=INF)
          ans[1]++;
   return ans;                         //返回出度和入度
}
```

程序验证

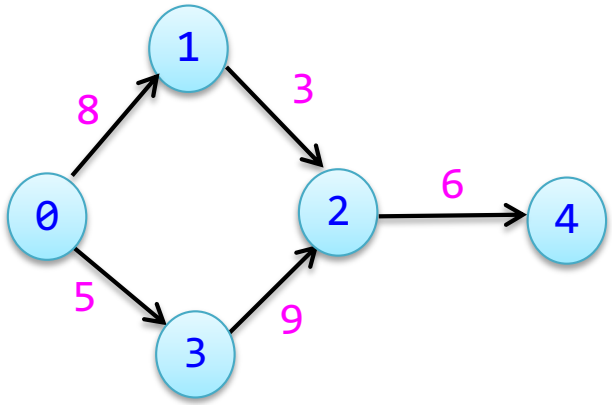
```
int main()
{  MatGraph g1,g2;
   int n=5,e=8;
   int a[MAXV][MAXV]={{0,1,0,1,1},{1,0,1,1,0},{0,1,0,1,1},
                      {1,1,1,0,1},{1,0,1,1,0}};
   g1.CreateMatGraph(a,n,e);
   printf("图G1(无向图)\n"); g1.DispMatGraph();
   printf("求解结果\n");
   for (int i=0;i<g1.n;i++)
       printf("  顶点%d的度: %d\n",i,Degree1(g1,i));

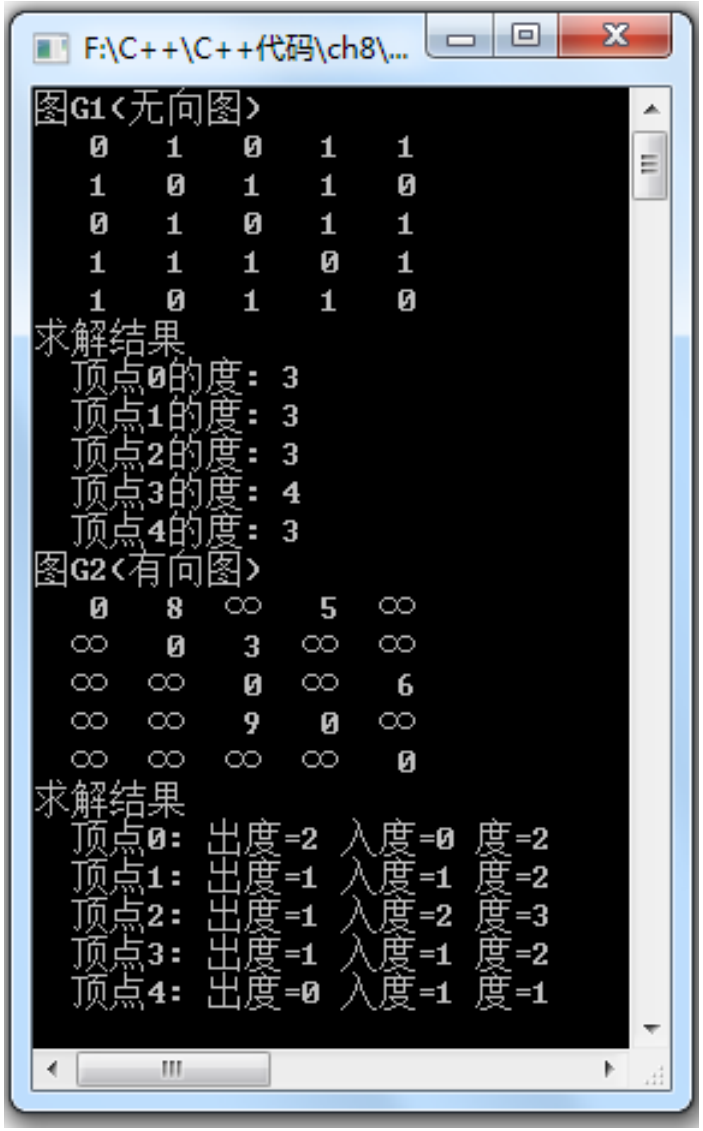
   n=5; e=5;
   int b[MAXV][MAXV]={{0,8,INF,5,INF},{INF,0,3,INF,INF},
       {INF,INF,0,INF,6},{INF,INF,9,0,INF},{INF,INF,INF,INF,0}};
   g2.CreateMatGraph(b,n,e);
   printf("图G2(有向图)\n"); g2.DispMatGraph();
   printf("求解结果\n");
   for (int i=0;i<g2.n;i++)
   {  vector<int> ans=Degree2(g2,i);
      printf("  顶点%d: 出度=%d 入度=%d 度=%d\n",
                        i,ans[0],ans[1],ans[0]+ans[1]);
   }
   return 0;
}
```

（a）一个无向图



（b）一个带权有向图

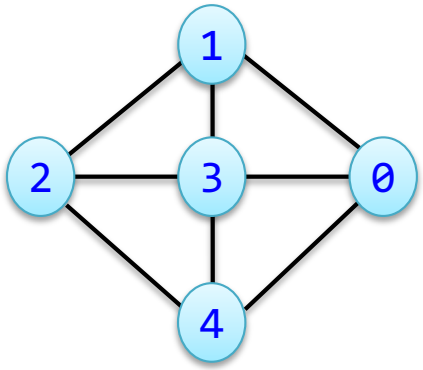




8.2.2 邻接表

1. 邻接表存储方法

- 对图中每个顶点i建立一个单链表，将顶点i的所有邻接点链起来。



顶点0的单链表



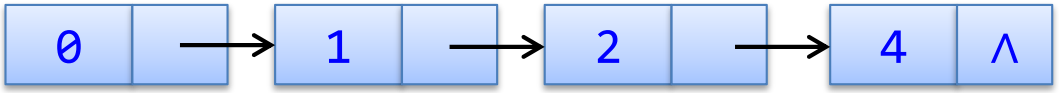
顶点1的单链表



顶点2的单链表



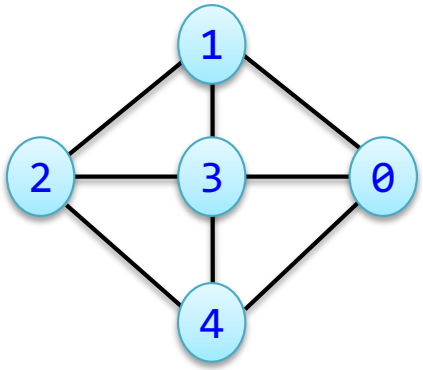
顶点3的单链表

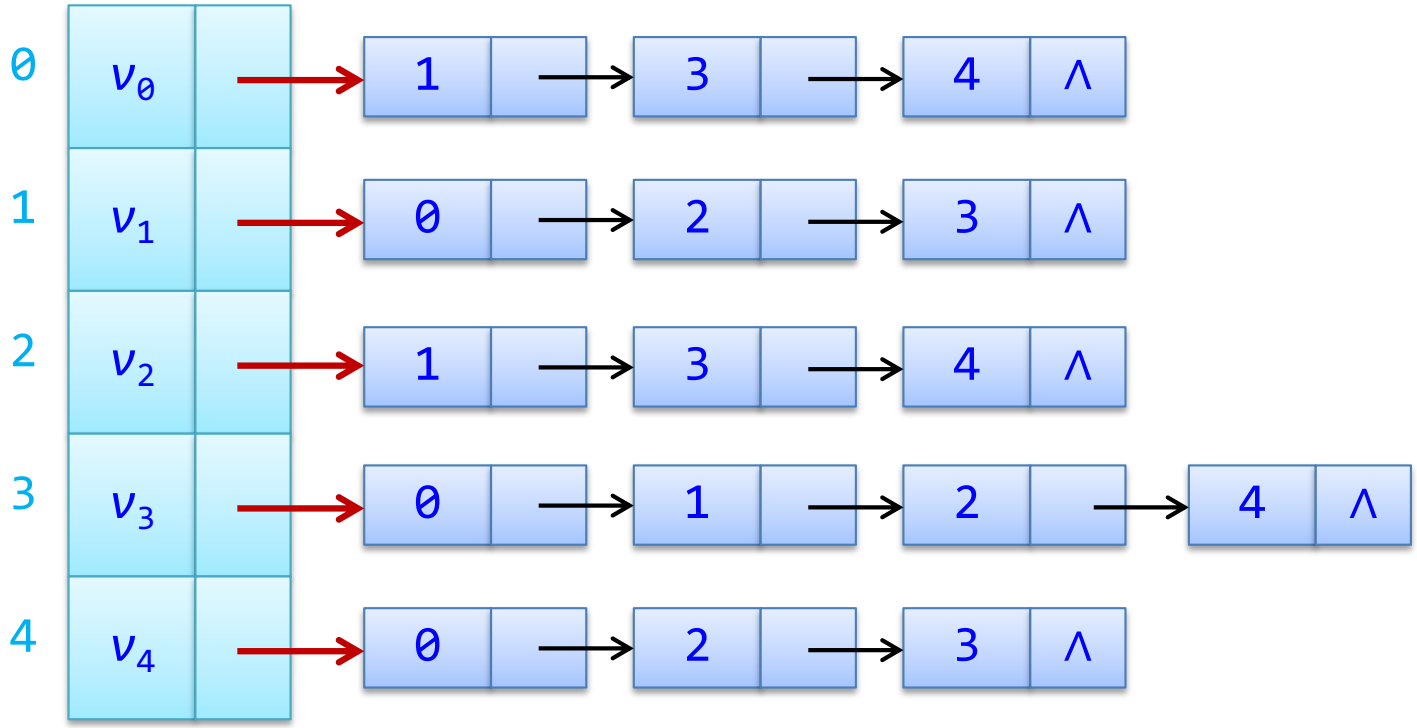


顶点4的单链表

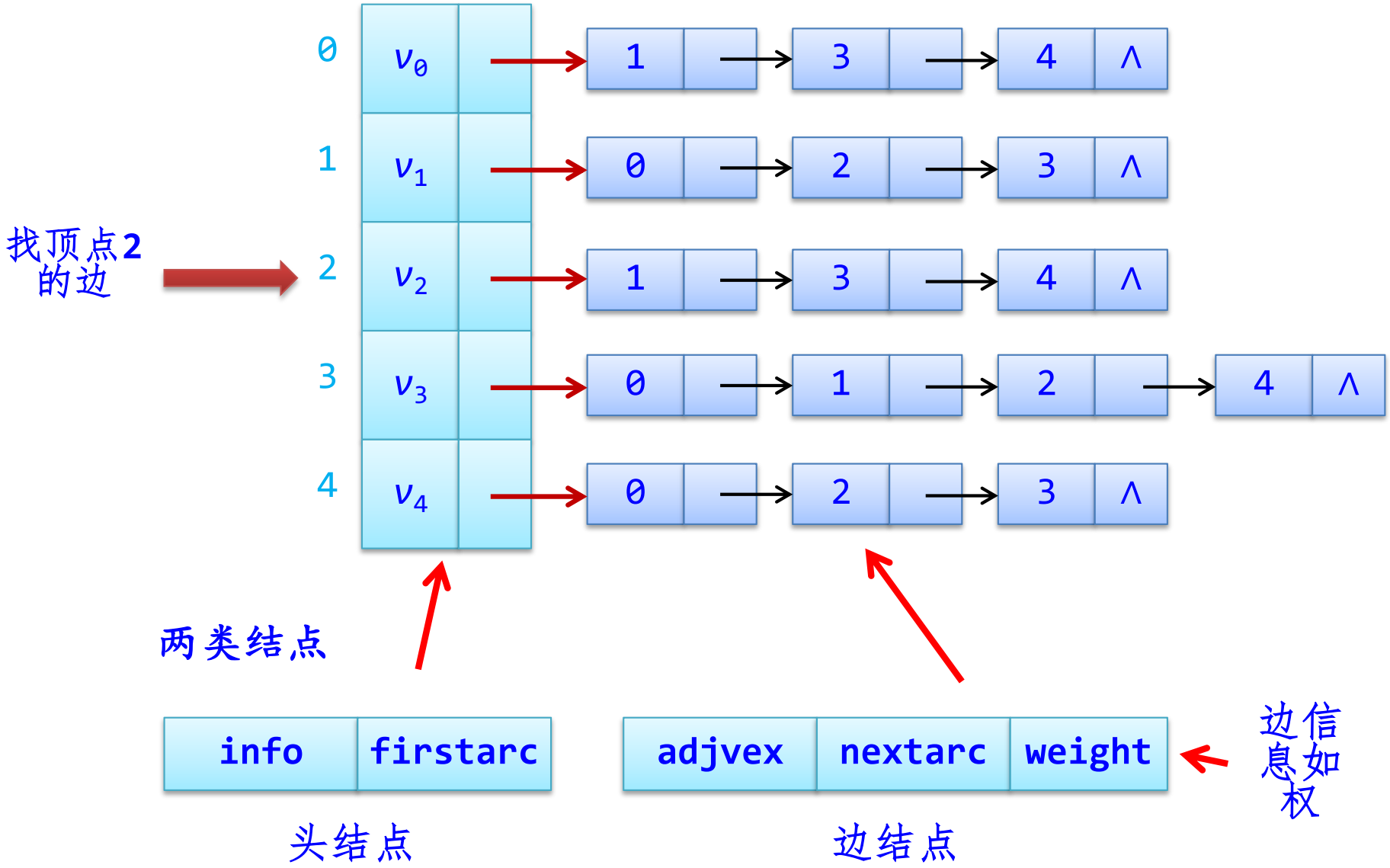


- 每个单链表上添加一个表头结点（表示顶点信息）。并将所有表头结点构成一个数组，下标为i的元素表示顶点i的表头结点。





图的邻接表存储方法是一种顺序分配与链式分配相结合的存储方法。



每个边结点的类型**ArcNode**定义如下

```
struct ArcNode                  //边结点类型
{   int adjvex;                 //邻接点
    int weight;                 //权值
    ArcNode* nextarc;           //指向下一条边的边结点
};
```

每个头结点的类型**HNode**定义如下

```
struct HNode                    //头结点类型
{   string info;                        //顶点信息
    ArcNode* firstarc;          //指向第一条边的边结点
};
```

图的邻接表存储类AdjGraph

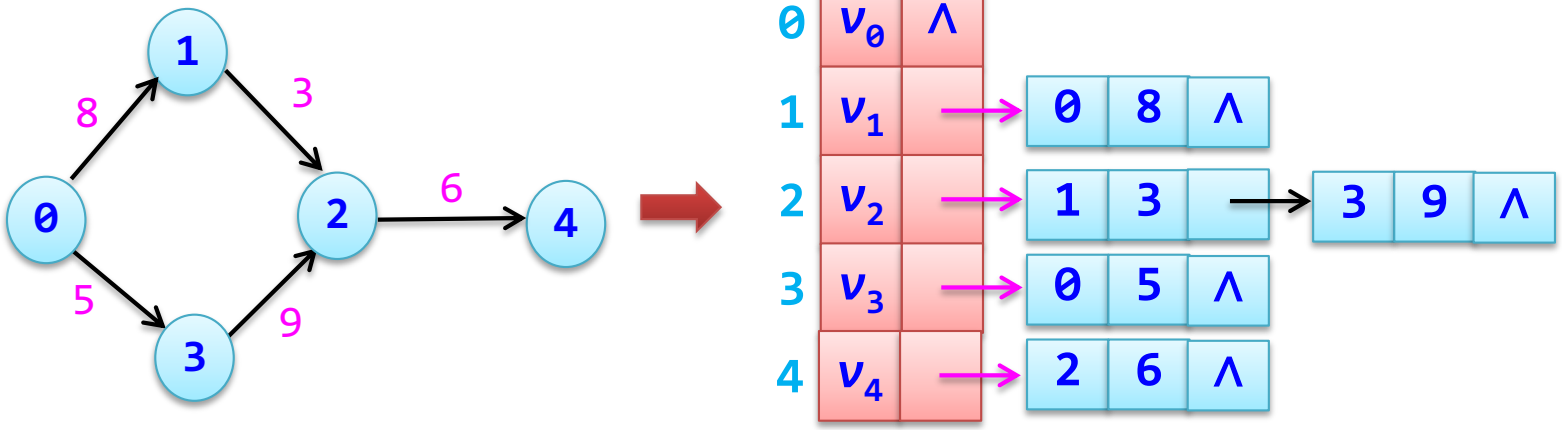
```
class AdjGraph                              //图邻接表类
{
public:
   HNode adjlist[MAXV];                     //头结点数组
   int n,e;                                 //顶点数，边数
   AdjGraph()                               //构造函数
   {  for (int i=0;i<MAXV;i++)              //头结点的firstarc置为空
         adjlist[i].firstarc=NULL;
   }
   ~AdjGraph()                              //析构函数,释放图的邻接表空间
   {  ArcNode* pre,*p;
      for (int i=0;i<n;i++)                 //遍历所有的头结点
      {  pre=adjlist[i].firstarc;
         if (pre!=NULL)
         {  p=pre->nextarc;
            while (p!=NULL)                 //释放adjlist[i]的所有边结点空间
            {  delete pre;
               pre=p; p=p->nextarc;         //pre和p指针同步后移
            }
            delete pre;
         }
      }
   }
   //图的基本运算算法
};
```

邻接表的特点

- 邻接表表示不唯一。
- 对于有n个顶点和e条边的无向图，其邻接表有n个表头结点和$2e$个边结点；对于有n个顶点和e条边的有向图，其邻接表有n个表头结点和e个边结点。显然，对于边数目较少的稀疏图，邻接表比邻接矩阵要节省空间。
- 对于无向图，顶点i（$0 \leqslant i \leqslant n-1$）对应的单链表的边结点个数正好是顶点$i$的度。
- 对于有向图，顶点i（$0 \leqslant i \leqslant n-1$）对应的单链表的边结点个数仅仅是顶点$i$的出度。顶点$i$的入度是邻接表中所有adjvex值为$i$的边结点个数。
- 用邻接表存储图时，确定任意两个顶点之间是否有边相连的时间为$O(m)$（m为最大顶点出度，$m<n$）。

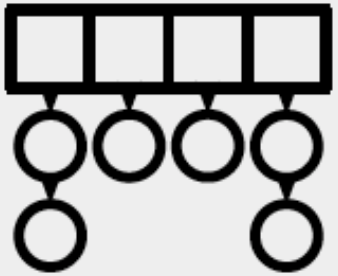
扩展

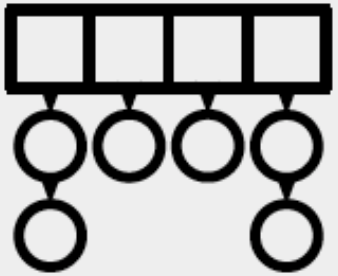
逆邻接表



一个带权有向图

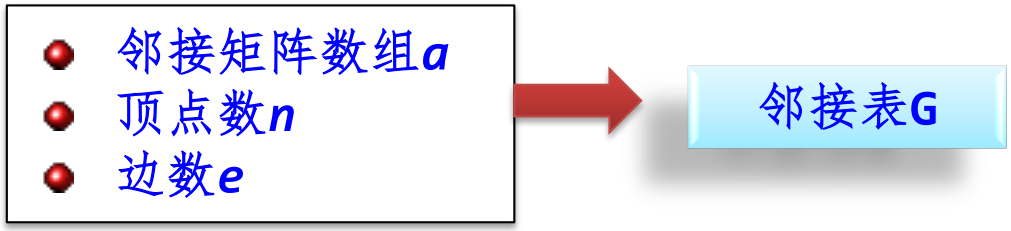
☎ 方便查找每个顶点的入边

For G = (V, E)	$\begin{bmatrix} 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 \end{bmatrix}$	
Edge Membership Is e = {u, v} in E?		
Neighbor Query What are the neighbors of u?		
Space requirements		

For G = (V, E)	$\begin{bmatrix} 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 \end{bmatrix}$	
Edge Membership Is e = {u, v} in E?	$O(1)$	$O(\deg(u))$ **or** $O(\deg(v))$
Neighbor Query What are the neighbors of u?	$O(\lvert V \rvert)$	$O(\deg(v))$
Space requirements	$O(\lvert V \rvert^2)$	$O(\lvert V \rvert + \lvert E \rvert)$

2．图基本运算在邻接表中的实现

（1）创建图的邻接表




```
void CreateAdjGraph(int a[][MAXV],int n,int e)
//通过a、n和e来建立图的邻接表
{  ArcNode* p;
   this->n=n; this->e=e;                          //置顶点数和边数
   for (int i=0;i<n;i++)                          //检查邻接矩阵中每个元素
      for (int j=n-1;j>=0;j--)
         if (a[i][j]!=0 && a[i][j]!=INF)          //存在一条边
         {  p=new ArcNode();                      //创建一个结点p
            p->adjvex=j;
            p->weight=a[i][j];
            p->nextarc=adjlist[i].firstarc; //采用头插法插入p
            adjlist[i].firstarc=p;
         }
}
```

（2）输出图

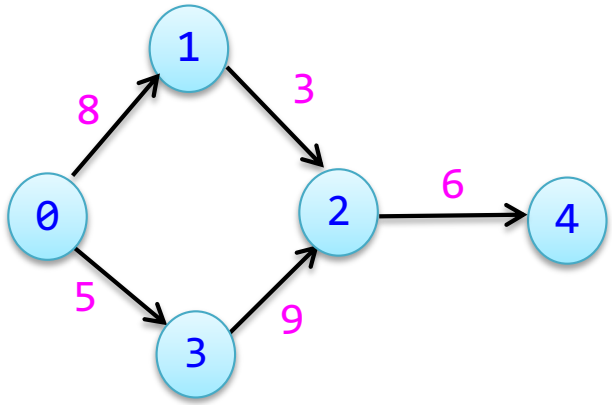
```
void DispAdjGraph()                          //输出图的邻接表
{  ArcNode* p;
   for (int i=0;i<n;i++)                     //遍历每个头结点
   {  printf("   [%d]",i);
      p=adjlist[i].firstarc;                 //p指向第一个邻接点
      if (p!=NULL)  printf("→");
      while (p!=NULL)                        //遍历第i个单链表
      {  printf(" (%d,%d)",p->adjvex,p->weight);
         p=p->nextarc;                       //p移向下一个邻接点
      }
      printf("\n");
   }
}
```

操作 将图的邻接表类型定义及其基本运算存放在**AdjGraph.cpp**文件中

程序验证

```
int main()
{   AdjGraph G;
    int n=5,e=5;
    int A[MAXV][MAXV]={{0,8,INF,5,INF},{INF,0,3,INF,INF},
        {INF,INF,0,INF,6},{INF,INF,9,0,INF},{INF,INF,INF,INF,0}};

    G.CreateAdjGraph(A,n,e);
    cout << "图的邻接表:\n"; G.DispAdjGraph();
    cout << "销毁图\n";
    return 0;
}
```



一个带权有向图



【例8.3】一个含有n个顶点e条边的图采用邻接表存储，设计以下算法：

（1）该图为无向图，求其中顶点v的度。

（2）该图为有向图，求该图中顶点v的出度和入度。



- 无向图，求其中顶点v的度

```
#include"AdjGraph.cpp"                    //包含图(邻接表)的基本运算算法
int Degree1(AdjGraph& G,int v)            //无向图邻接表G中求顶点v的度
{  int d=0;
   ArcNode* p=G.adjlist[v].firstarc;
   while (p!=NULL)                        //统计单链表v中边结点个数
   {  d++;
      p=p->nextarc;
   }
   return d;
}
```

- 有向图，求该图中顶点v的出度和入度

```
vector<int> Degree2(AdjGraph& G,int v)
//有向图邻接表G中求顶点v的出度和入度
{  vector<int> ans={0,0};                    //ans[0]累计出度,ans[1]累计入度
   ArcNode* p=G.adjlist[v].firstarc;
   while (p!=NULL)                           //统计单链表v中边结点个数
   {  ans[0]++;
      p=p->nextarc;
   }
   for (int i=0;i<G.n;i++)                   //统计所有为v的边结点个数为v的入度
   {  p=G.adjlist[i].firstarc;
      while (p!=NULL)
      {  if (p->adjvex==v)
         {  ans[1]++;
            break;                           //一个单链表最多只有一个这样的结点
         }
         p=p->nextarc;
      }
   }
   return ans;                               //返回出度和入度
}
```

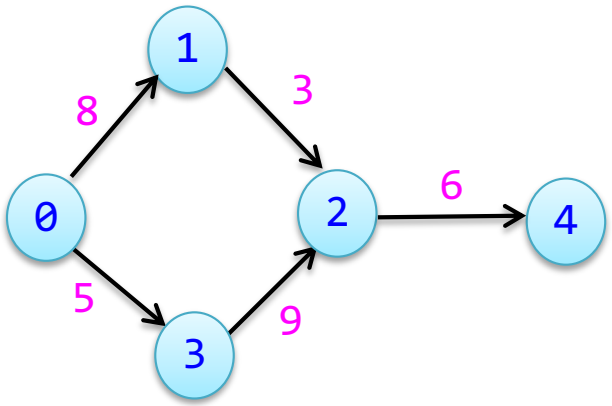
3．简化的邻接表

（1）简化邻接表 I

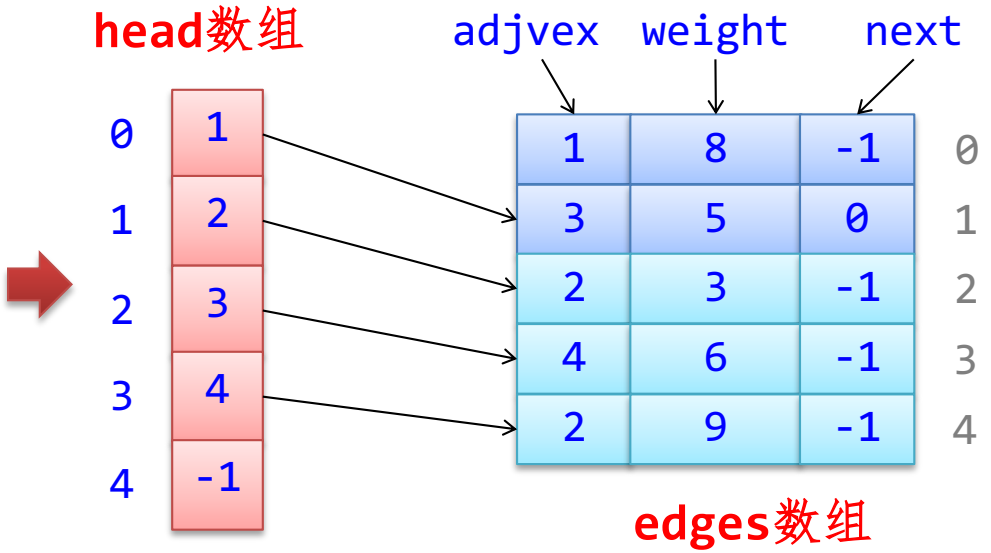
- 直接用两个数组表示邻接表，头结点数组为head。
- 边结点数组edges为ENode类型，该类型包含adjvex、weight和next成员变量，其中head[i]表示顶点i的单链表（head[i]=-1表示顶点i没有出边）。

```
int head[MAXV];                 //头结点数组
struct Edge                     //边结点类型
{   int adjvex;                 //邻接点
    int weight;                 //权值
    int next;                   //下一个边结点在edges数组中的下标
} edges[MAXE];                  //边结点数组
int n;                          //顶点数
int cnt;                        //edges数组元素个数
```

```
int head[MAXV];                    //头结点数组
struct Edge                        //边结点类型
{   int adjvex;                    //邻接点
    int weight;                    //权值
    int next;                      //下一个边结点在edges数组中的下标
} edges[MAXE];                     //边结点数组
int n;                             //顶点数
int cnt;                           //edges数组元素个数
```



一个带权有向图



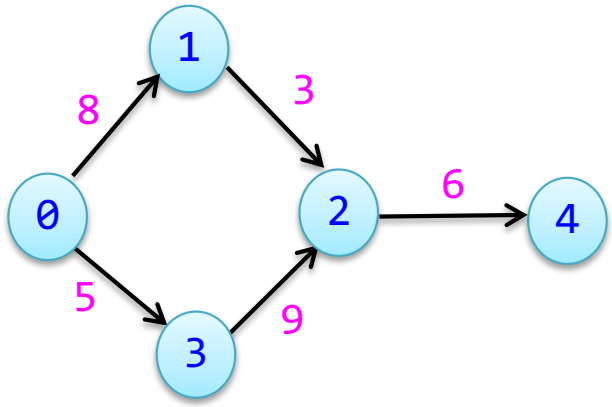
```
void init()                                  //初始化
{   cnt=0;                                   //cnt从0开始
    memset(head,0xff,sizeof(head));          //所有元素初始化为-1
}

void addedge(int u,int v,int w)              //添加一条有向边<u,v>:w
{   edges[cnt].adjvex=v;                     //该边插入到edges数组末尾
    edges[cnt].weight=w;
    edges[cnt].next=head[u]; //将edges[cnt]边结点插入到head[u]的表头
    head[u]=cnt;
    cnt++;                                   //edges数组元素个数增1
}
```

（2）简化邻接表II

- 直接使用vector向量Adj存放邻接表。
- 其中Adj[i]向量存放顶点i的所有出边结点，每个出边结点表示为“[出边邻接点编号，权值]”，不含next指针，直接将所有的出边邻接点构成一个向量。

```
struct Edge                 //边结点类型
{   int adjvex;             //邻接点编号
    int weight;             //权值
};
vector<vector<Edge>> Adj;
Adj.resize(MAXV);           //Adj向量中添加MAXV个{}的元素
```



一个带权有向图

```
Adj={{[3, 5], [1, 8]}, {[2, 3]}, {[4, 6]}, {[2, 9]}, {}}
```

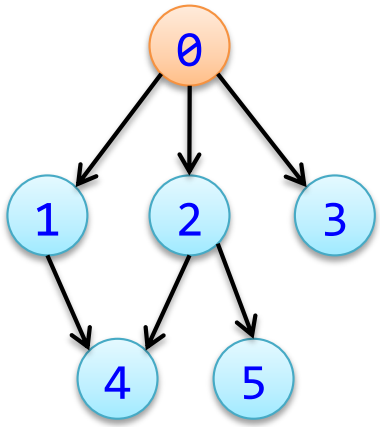
Adj[0]

8.3 图的遍历

8.3.1 图遍历的概念

- 从给定图中任意指定的顶点（称为初始点）出发，按照某种搜索方法沿着图的边访问图中的所有顶点，使每个顶点仅被访问一次，这个过程称为图遍历。

- 为了避免同一个顶点被重复访问，必须记住访问过的顶点。为此，可设置一个访问标志数组**visited**，初始时所有元素置为**0**，当顶点**i**访问过时，该数组元素**visited[i]**置为**1**。
- 根据遍历方式的不同，图的遍历方法有两种：一种是深度优先遍历（**DFS**）方法；另一种是广度优先遍历（**BFS**）方法。



DFS: 0 1 4 2 5 3

BFS: 0 1 2 3 4 5

8.3.2 深度优先遍历

- 从图中某个起始点v出发进行深度优先搜索–DFS(v)，首先访问初始顶点v。
- 然后选择一个与顶点v邻接且没被访问过的顶点w为初始顶点，再从w出发进行深度优先搜索–DFS(w)，直到图中与当前顶点v邻接的所有顶点都被访问过为止。

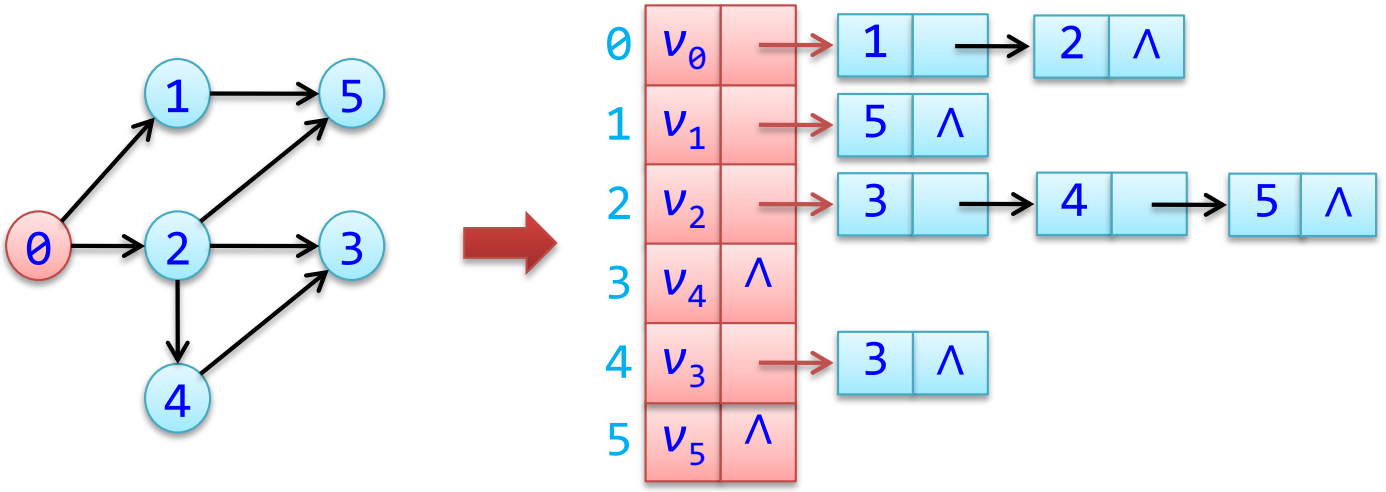
递归过程

图采用邻接表为存储结构，其深度优先遍历算法如下（其中，v是起始点编号，visited是全局数组）：

```
int visited[MAXV];                          //全局数组
void DFS(AdjGraph& G,int v)                 //深度优先遍历(邻接表)
{  cout << v << " ";                        //访问顶点v
   visited[v]=1;                            //置已访问标记
   ArcNode*p=G.adjlist[v].firstarc;         //p指向顶点v的第一个邻接点
   while (p!=NULL)
   {  int w=p->adjvex;                      //邻接点为w
      if (visited[w]==0) DFS(G,w);          //若w顶点未访问,递归访问它
      p=p->nextarc;                         //p置为下一个邻接点
   }
}
```

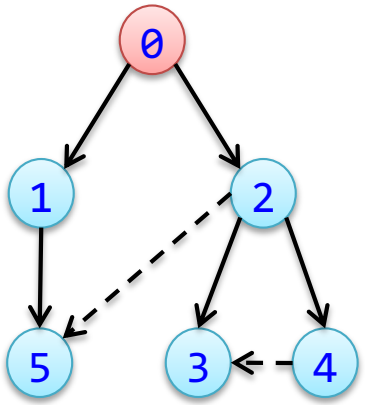
时间复杂度为$O(n+e)$。

理解深度优先遍历过程




```
int visited[MAXV];                          //全局数组
void DFS(AdjGraph& G,int v)                 //深度优先遍历(邻接表)
{   cout << v << " ";                       //访问顶点v
    visited[v]=1;                           //置已访问标记
    ArcNode*p=G.adjlist[v].firstarc;        //p指向顶点v的第一个邻接点
    while (p!=NULL)
    {   int w=p->adjvex;                    //邻接点为w
        if (visited[w]==0) DFS(G,w);        //若w顶点未访问,递归访问它
        p=p->nextarc;                       //p置为下一个邻接点
    }
}
```

理解深度优先遍历过程



起始点0到图中其他顶点的路径长度：

0

0 → 1

0 → 1 → 5

0 → 2

0 → 2 → 5

0 → 2 → 3

0 → 2 → 4

0 → 2 → 4 → 3

DFS: 0 1 5 2 3 4

说明

类似树的先根遍历。

图采用邻接矩阵为存储结构，其深度优先遍历算法如下（其中，v是起始点编号，visited是全局数组）：

```
void DFS(MatGraph& g,int v)   //深度优先遍历(邻接矩阵)
{  cout << v << " ";          //访问访问v
   visited[v]=1;                        //置已访问标记
   for (int w=0;w<g.n;w++)
   {  if (g.edges[v][w]!=0 && g.edges[v][w]!=INF)
      {  if (visited[w]==0)   //存在边<v,w>并且w没有访问过
            DFS(g,w);         //若w顶点未访问,递归访问它
      }
   }
   visited[v]=0;
}
```

时间复杂度为$O(n^2)$。

8.3.3 广度优先遍历

- 首先访问起始点v。
- 接着访问顶点v的所有未被访问过的邻接点v_1、v_2、…、v_t。
- 然后再按照v_1、v_2、…、v_t的次序，访问每一个顶点的所有未被访问过的邻接点。
- 依次类推，直到图中所有和初始点v有路径相通的顶点或者图中所有已访问顶点的邻接点都被访问过为止。

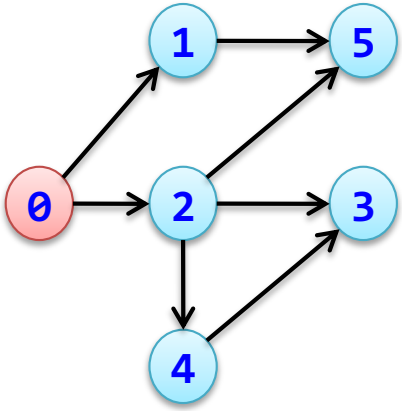


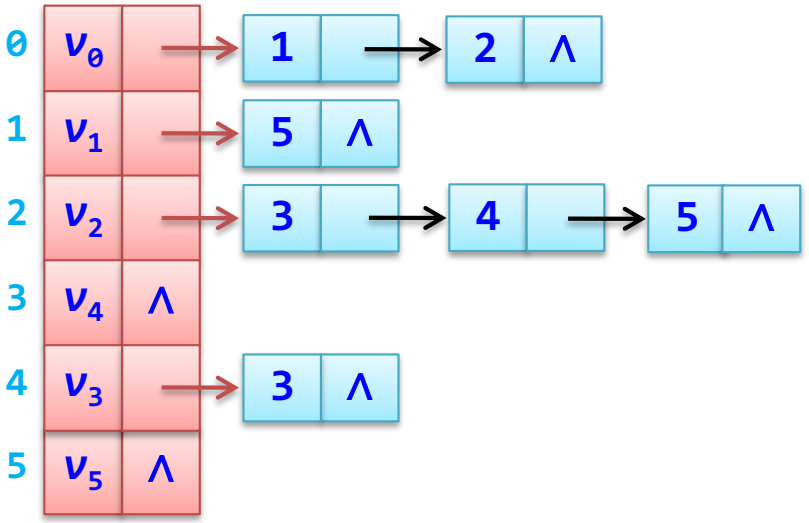
相邻顶点：先访问先处理  **队列**

图采用邻接表为存储结构，其广度优先遍历算法如下：

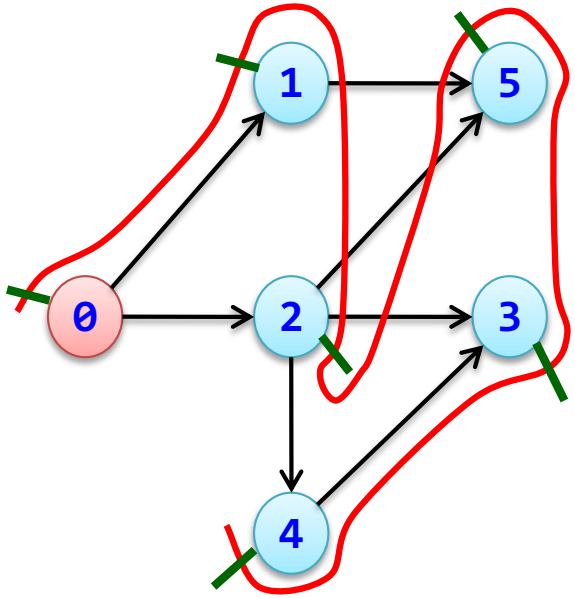
```
void BFS(AdjGraph& G,int v)              //广度优先遍历(邻接表)
{  int visited[MAXV];
   memset(visited,0,sizeof(visited)); //初始化visited数组
   queue<int> qu;                        //定义一个队列
   cout << v << " ";                     //访问顶点v
   visited[v]=1;                         //置已访问标记
   qu.push(v);                           //顶点v进队
   while (!qu.empty())                   //队列不空循环
   {  int u=qu.front(); qu.pop();        //出队顶点u
      ArcNode* p=G.adjlist[u].firstarc; //找顶点u的第一个邻接点
      while (p!=NULL)
      {  if (visited[p->adjvex]==0)      //若u的邻接点未访问
         {  cout << p->adjvex << " ";    //访问邻接点
            visited[p->adjvex]=1;        //置已访问标记
            qu.push(p->adjvex);          //邻接点进队
         }
         p=p->nextarc;                   //找下一个邻接点
      }
   }
}
```

时间复杂度为$O(n+e)$。



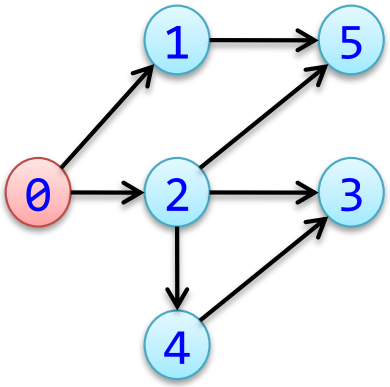


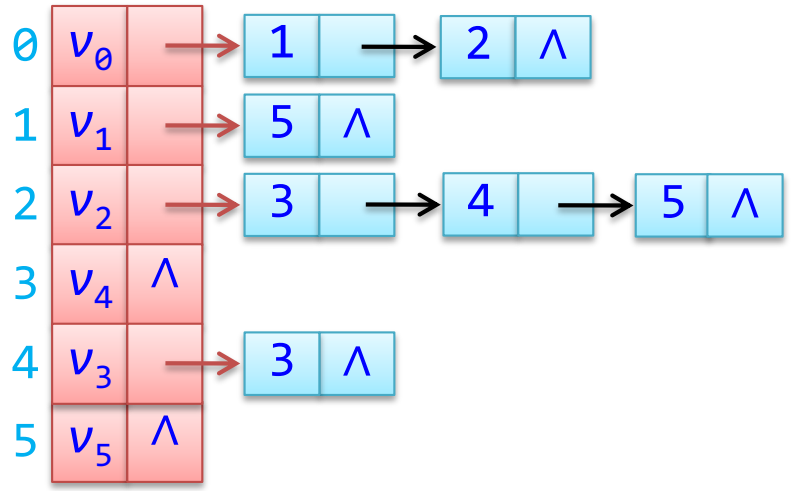
（a）一个有向图G_4　　（b）图G_4的邻接表

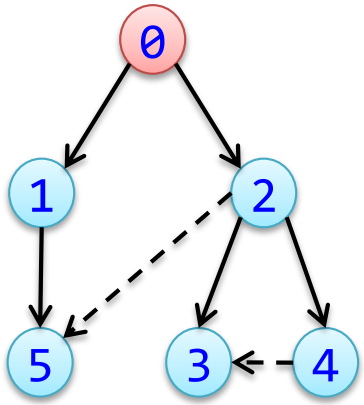


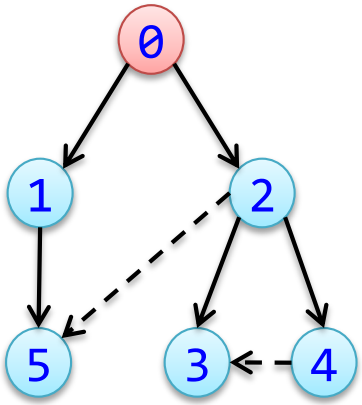
BFS: 0 1 2 5 3 4

理解广度优先遍历过程









起始点0到图中其他顶点的路径长度：

0

0 → 1

0 → 2

0 → 1 → 5

0 → 2 → 5

0 → 2 → 3

0 → 2 → 4

0 → 2 → 4 → 3

BFS: 0 1 2 5 3 4

说明

类似树的层次遍历。

图采用邻接矩阵为存储结构，其广度优先遍历算法如下：

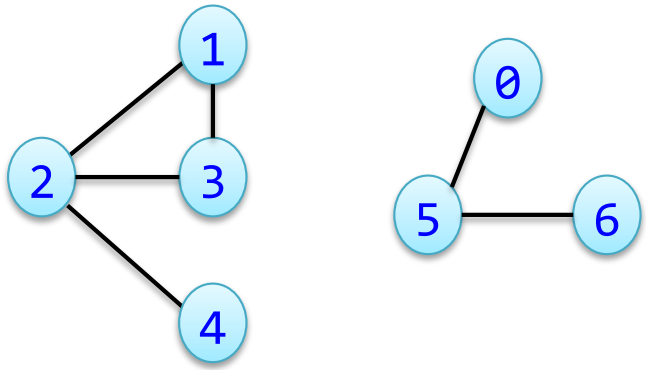
```
void BFS(MatGraph& g,int v)                    //广度优先遍历(邻接矩阵)
{  int visited[MAXV];
   memset(visited,0,sizeof(visited));          //初始化visited数组
   queue<int> qu;                              //定义一个队列
   cout << v << " ";                           //访问顶点v
   visited[v]=1;                               //置已访问标记
   qu.push(v);                                 //顶点v进队
   while (!qu.empty())                         //队列不空循环
   {  int u=qu.front(); qu.pop();              //出队顶点u
      for (int i=0;i<g.n;i++)
         if (g.edges[u][i]!=0 && g.edges[u][i]!=INF)
         {  if (visited[i]==0)                 //存在边<u,i>并且顶点i未访问
            {  cout << i << " ";               //访问邻接点i
               visited[i]=1;                   //置已访问标记
               qu.push(i);                     //邻接点i进队
            }
         }
   }
}
```

时间复杂度为$O(n^2)$。

8.3.4 非连通图的遍历

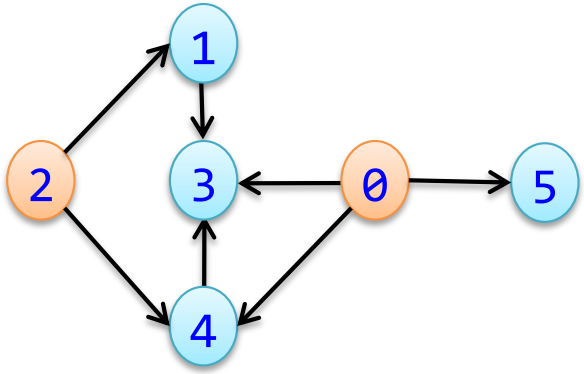
无向图

- 连通图：一次遍历能够访问到图中的所有顶点。
- 非连通图：一次遍历只能访问到起始点所在连通分量中的所有顶点，其他连通分量中的顶点是不可能访问到的。为此需要从其他每个连通分量中选择起始点，分别进行遍历，才能够访问到图中的所有顶点。



有向图

- 有向图，若从起始点到图中的其他每个顶点都有路径，则能够访问到图中的所有顶点。
- 否则不能访问到所有顶点，为此同样需要再选起始点，继续进行遍历，直到图中的所有顶点都被访问过为止。



非连通图采用邻接表存储结构，其深度优先遍历算法如下：

```
void DFSA(AdjGraph& G)                    //非连通图的DFS
{  for (int i=0;i<G.n;i++)
      if (visited[i]==0)                  //若顶点i没有访问过
         DFS(G,i);                        //从顶点i出发深度优先遍历
}
```

非连通图采用邻接表存储结构，其广度优先遍历算法如下：

```
void BFSA(AdjGraph& G)                    //非连通图的BFS
{  for (int i=0;i<G.n;i++)
      if (visited[i]==0)                  //若顶点i没有访问过
         BFS(G,i);                        //从顶点i出发广度优先遍历
}
```

【例8.4】假设图采用邻接表存储，设计一个算法，判断一个无向图是否连通。若连通则返回true；否则返回false。



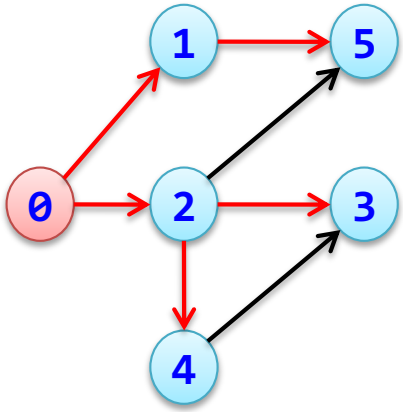

```
int visited[MAXV];                          //全局数组
void DFS(AdjGraph& G,int v)                 //深度优先遍历(邻接表)
{  visited[v]=1;                            //置已访问标记（不输出v）
   ArcNode*p=G.adjlist[v].firstarc;         //p指向顶点v的第一个邻接点
   while (p!=NULL)
   {  int w=p->adjvex;                      //邻接点为w
      if (visited[w]==0)
         DFS(G,w);                          //若w顶点未访问,递归访问它
      p=p->nextarc;                         //p置为下一个邻接点
   }
}
```

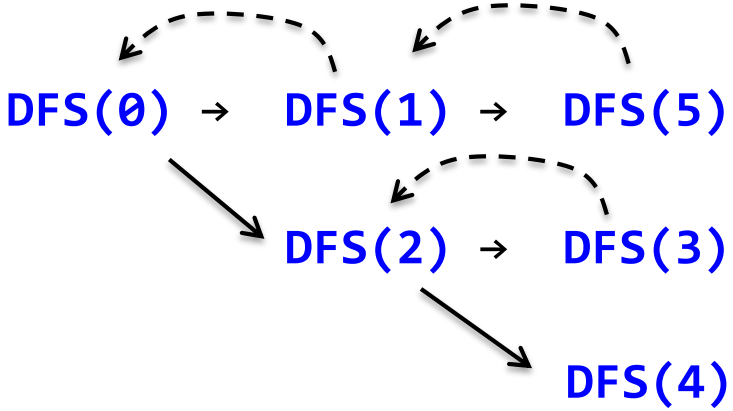
判断一个无向图是否连通。若连通则返回true；否则返回false。

```
bool Connect(AdjGraph& G)                    //判断无向图G的连通性
{  memset(visited,0,sizeof(visited));
   DFS(G,0);                                 //从0出发深度优先遍历
   for (int i=0;i<G.n;i++)
       if (visited[i]==0)                    //若有顶点没有访问过
          return false;                      //说明是非连通图
   return true;                              //所有顶点均访问说明是连通图
}
```

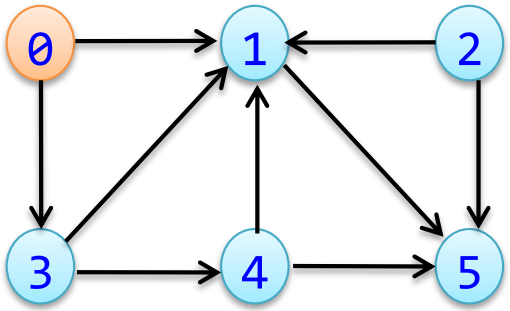
8.4 图遍历算法的应用

8.4.1 深度优先遍历算法的应用

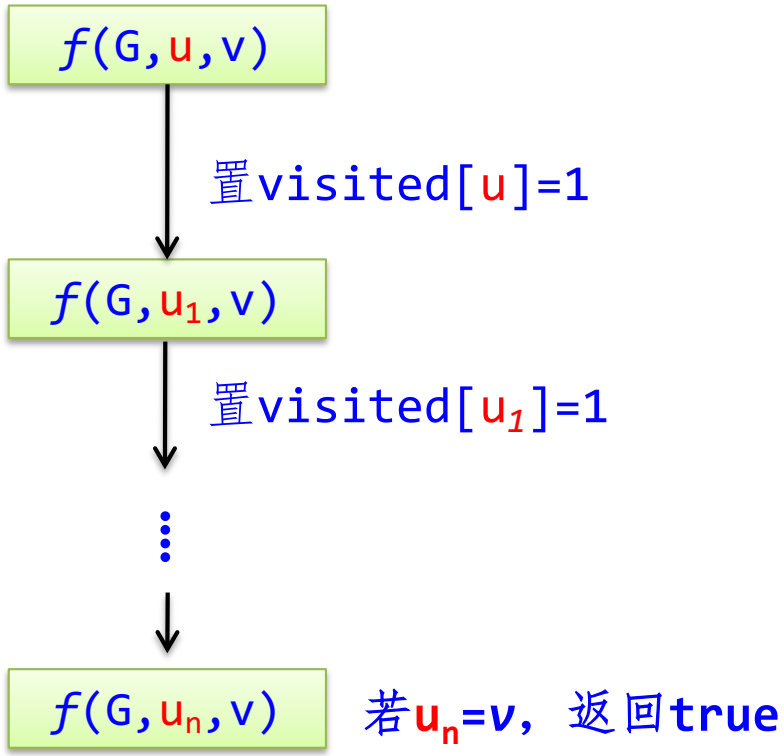




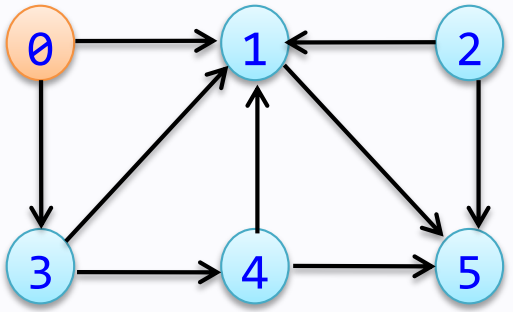
【例8.5】假设图G采用邻接表存储，设计一个算法判断顶点u到顶点v之间是否有路径。并对于以下有向图，判断从顶点0到顶点5、从顶点0到顶点2是否有路径。

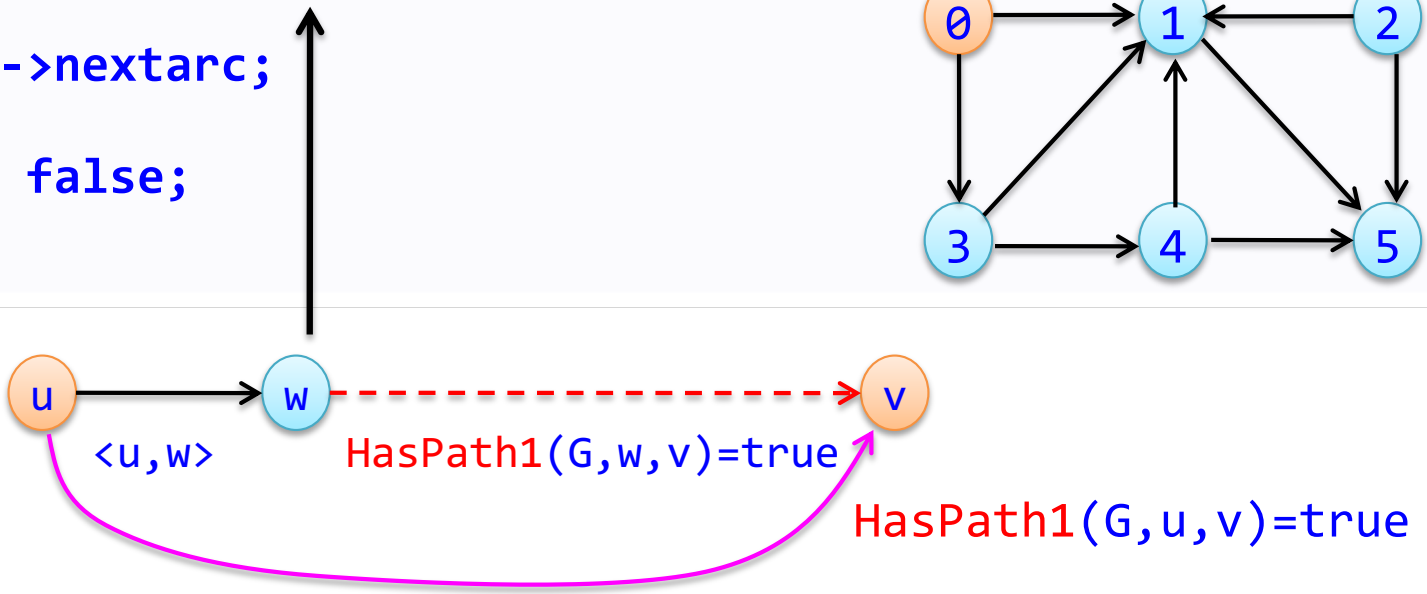


思路



```
#include"AdjGraph.cpp"          //包含图(邻接表)的基本运算算法
#include<cstring>
int visited[MAXV];                                  //全局数组
bool HasPath1(AdjGraph& G,int u,int v)              //被HasPath方法调用
{  visited[u]=1;
   ArcNode* p=G.adjlist[u].firstarc;
   while (p!=NULL)
   {  int w=p->adjvex;                              //找到u的邻接点w
      if (w==v)                                     //找到目标点后返回真
         return true;                               //表示u到v有路径
      else if (visited[w]==0)                       //若顶点w没有访问
      {  if (HasPath1(G,w,v))                       //找到路径<u,w>+w→v
            return true;
      }
      p=p->nextarc;
   }
   return false;
}
```

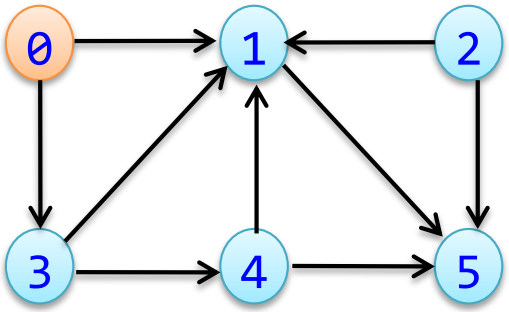




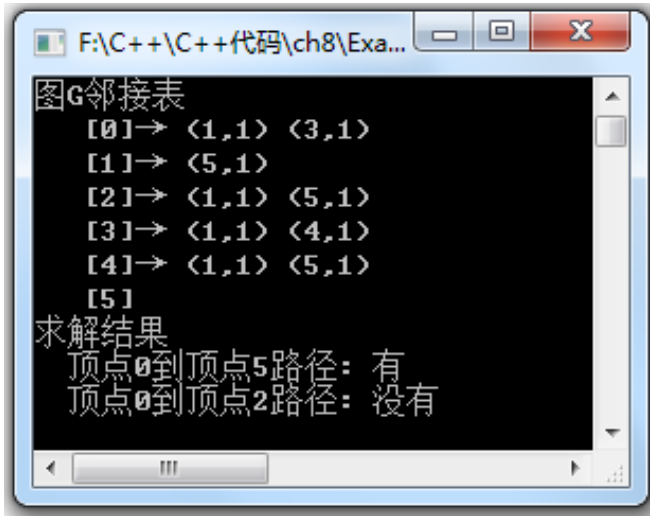
程序验证

```
bool HasPath(AdjGraph& G,int u,int v)  //判断u到v是否有简单路径
{  memset(visited,0,sizeof(visited));
   return HasPath1(G,u,v);
}

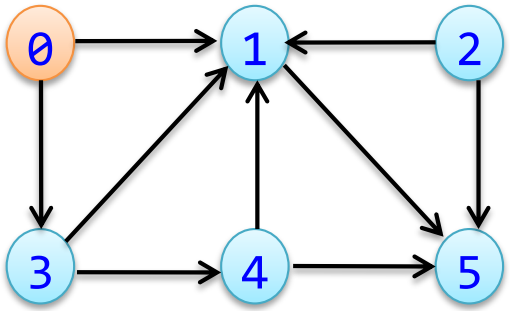
int main()
{  AdjGraph G;
   int n=6,e=9;
   int a[MAXV][MAXV]={{0,1,0,1,0,0},{0,0,0,0,0,1},{0,1,0,0,0,1},
                      {0,1,0,0,1,0},{0,1,0,0,0,1},{0,0,0,0,0,0}};
   G.CreateAdjGraph(a,n,e);
   printf("图G邻接表\n"); G.DispAdjGraph();
   int u=0,v=5;
   printf("求解结果\n");
   printf("  顶点%d到顶点%d路径: %s\n",u,v,HasPath(G,u,v)?"有":"没有");
   u=0; v=2;
   printf("  顶点%d到顶点%d路径: %s\n",u,v,HasPath(G,u,v)?"有":"没有");
   return 0;
}
```


0
1
2
3
4
5



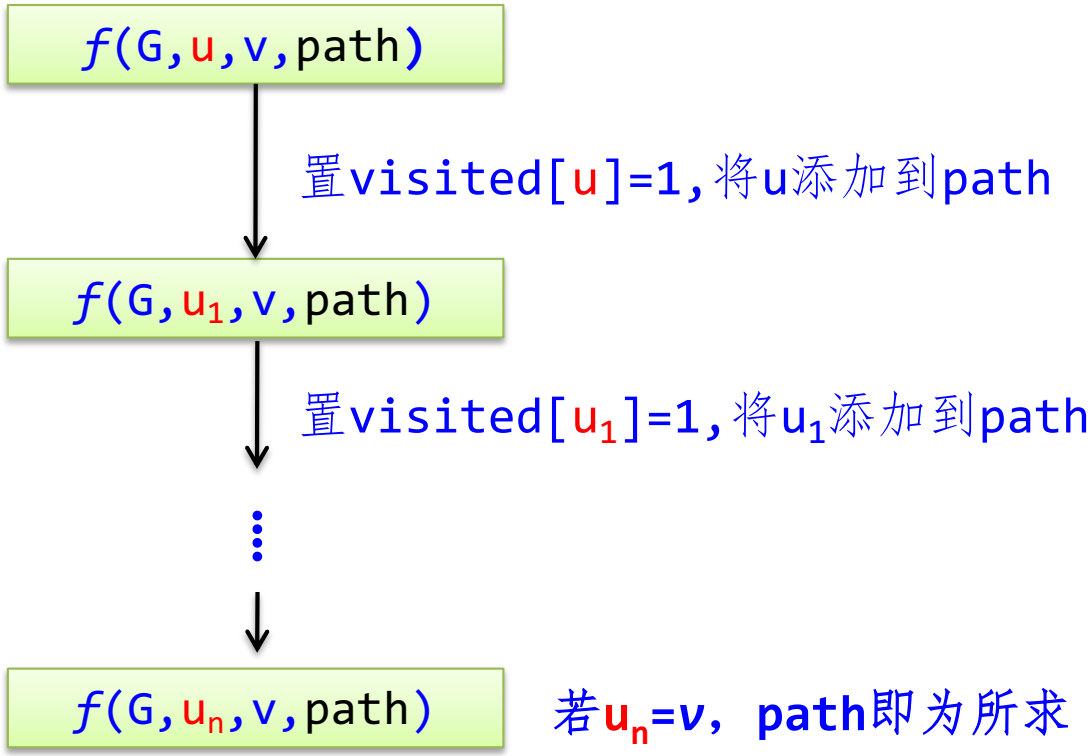

F:\C++\C++代码\ch8\Exa...
图G邻接表
[0]→ (1,1) (3,1)
[1]→ (5,1)
[2]→ (1,1) (5,1)
[3]→ (1,1) (4,1)
[4]→ (1,1) (5,1)
[5]
求解结果
顶点0到顶点5路径: 有
顶点0到顶点2路径: 没有

【例8.6】假设图G采用邻接表存储，设计一个算法求顶点*u*到顶点*v*之间的一条简单路径（假设两顶点之间存在一条或多条简单路径）。并对于以下有向图，求从顶点0到顶点4的一条简单路径。



思路

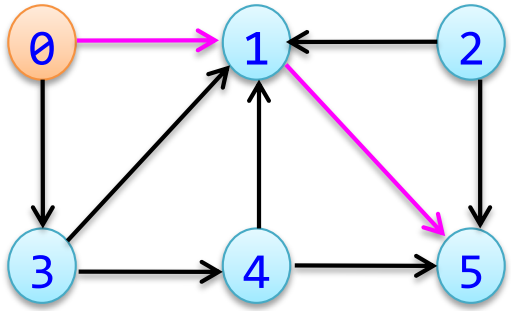
vector向量path存放一条路径



path是非引用参数，它具有自动回退功能

```
int visited[MAXV];                          //全局数组
void FindaPath11(AdjGraph& G,int u,int v,vector<int> path)
//被FindaPath1调用
{  visited[u]=1;
   path.push_back(u);                       //顶点u加入到路径中
   if (u==v)                                //找到一条路径后输出并返回
   {  for (int i=0;i<path.size();i++)
          printf("%d ",path[i]);
      printf("\n");
      return;
   }
   ArcNode* p=G.adjlist[u].firstarc;
   while (p!=NULL)
   {  int w=p->adjvex;                      //找到u的邻接点w
      if (visited[w]==0)                    //若顶点w没有访问
         FindaPath11(G,w,v,path);           //从w出发继续查找
      p=p->nextarc;
   }
}
```

```
void FindaPath1(AdjGraph& G,int u,int v)
//求u到v的一条简单路径:直接输出
{  memset(visited,0,sizeof(visited));
   vector<int> path;                    //path存放搜索路径
   FindaPath11(G,u,v,path);
}
```



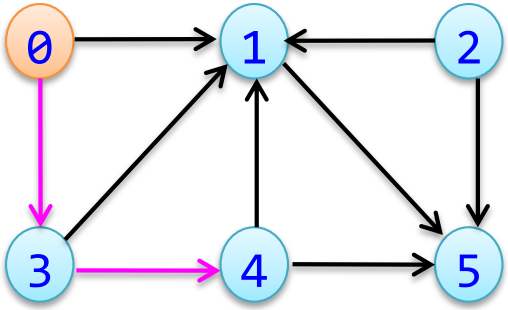
FindaPath1(G,0,5)

顶点0到顶点5路径：0 1 5

如果不是直接输出路径而是返回找到的路径呢？此时需要增加一个与path同类型的引用参数res，一旦找到路径置res为path。

```
void FindaPath21(AdjGraph& G,int u,int v,vector<int> path,
                                          vector<int>& res)
//被FindaPath2调用
{  visited[u]=1;
   path.push_back(u);                          //顶点u加入到路径中
   if (u==v)                                   //找到一条路径后返回
   {  res=path;
      return;
   }
   ArcNode* p=G.adjlist[u].firstarc;
   while (p!=NULL)
   {  int w=p->adjvex;                         //找到u的邻接点w
      if (visited[w]==0)                       //若顶点w没有访问
         FindaPath21(G,w,v,path,res);          //从w出发继续查找
      p=p->nextarc;
   }
}
```

```
void FindaPath2(AdjGraph& G,int u,int v)
//求u到v的一条简单路径:返回后输出
{  memset(visited,0,sizeof(visited));
   vector<int> path;                          //path存放搜索路径
   vector<int> res;                           //res存放找到的一条路径
   FindaPath21(G,u,v,path,res);               //求res
   for (int i=0;i<res.size();i++)             //输出res
      printf("%d ",res[i]);
   printf("\n");
}
```



FindaPath2(G,0,4)

顶点0到顶点4路径: 0 3 4

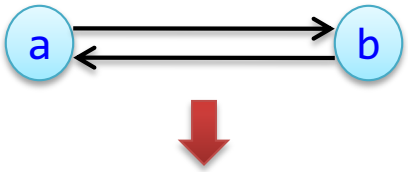



```
FindaPath21(AdjGraph& G,int u,int v,vector<int> path,
                                    vector<int>& res)
```

path和res的差别？

- path是非引用参数，它具有自动回退功能。找到顶点v时存放的恰好是u到v的路径。
- res是引用参数，它具有回传参数的功能（相当于全局变量）。找到顶点v时置res=path，这样res恰好是u到v的路径。
- 两种结合返回一条u到v的路径。

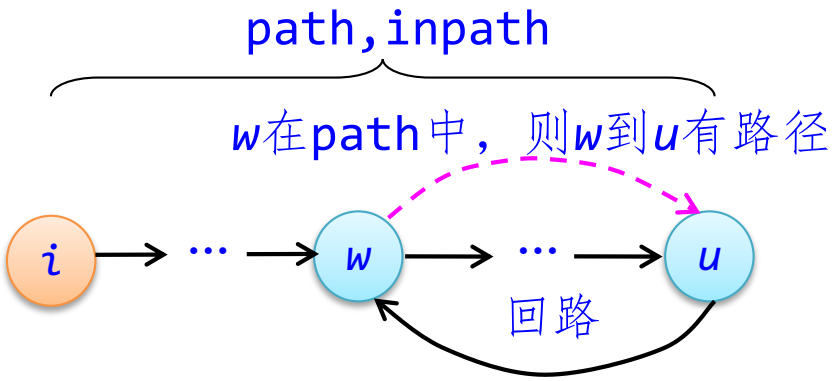
【例8.7】假设有向图G采用邻接表存储，设计一个算法，判断图中是否包含任何简单回路。注意若一个有向图只有两个顶点*a*和*b*，并且存在边<*a*，*b*>和<*b*，*a*>，则认为存在回路。



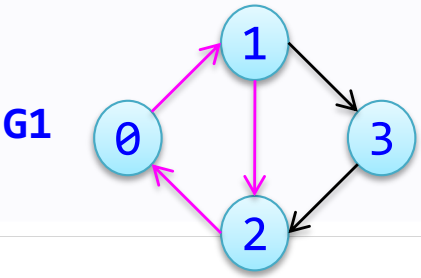
存在回路

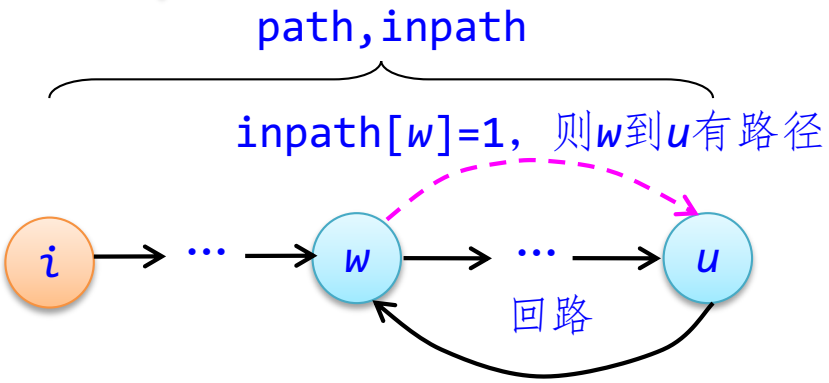


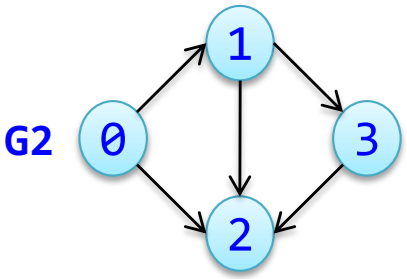
- 对于给定的有向图，从任意顶点**i**出发深度优先遍历，当走到顶点**u**时，**path**中保存从**v**到**u**的路径，**inpath**数组记录一个顶点是否在**path**中。
- 若**u**的一个邻接点**w**是**path**中的一个顶点，则构成一个包含顶点**u**和**w**的回路。



```
bool Cycle1(AdjGraph& G,int u,vector<int> path,vector<int> inpath)
//有向图G中从顶点u出发搜索是否存在回路的算法
{  path.push_back(u);                          //将顶点u添加到路径中
   inpath[u]=1;                                //置已访问标记
   ArcNode* p=G.adjlist[u].firstarc; //p指向顶点u的第一个邻接点
   while (p!=NULL)
   {  int w=p->adjvex;
      if (inpath[w]==0)                        //若顶点w不在路径中
      {  if (Cycle1(G,w,path,inpath))//从顶点w出发搜索存在回路
            return true;
      }
      else return true;                        //若顶点w在路径中，则出现回路
      p=p->nextarc;                            //找下一个邻接点
   }
   return false;
}
```





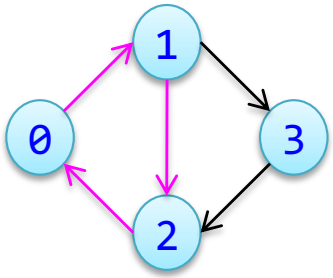


```
bool Cycle(AdjGraph& G)                    //判断有向图G中是否有回路
{  for (int i=0;i<G.n;i++)
   {  vector<int> inpath(MAXV,0);          //所有元素初始为0
      vector<int> path;
      if (Cycle1(G,i,path,inpath))         //从顶点i出发搜索
         return true;
   }
   return false;
}
```

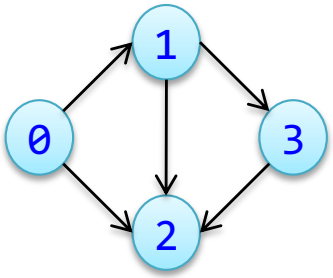
程序验证

```
int main()
{   AdjGraph G1;
    int n=4,e=5;
    int a[MAXV][MAXV]={{0,1,0,0},{0,0,1,1},{1,0,0,0},{0,0,1,0}};
    G1.CreateAdjGraph(a,n,e);
    printf("图G1邻接表\n"); G1.DispAdjGraph();
    printf("求解结果\n");
    printf("    图G1中是否有回路: %s\n",Cycle(G1)==true?"有":"没有");
    printf("\n");
    AdjGraph G2;
    n=4; e=5;
    int b[MAXV][MAXV]={{0,1,1,0},{0,0,1,1},{0,0,0,0},{0,0,1,0}};
    G2.CreateAdjGraph(b,n,e);
    printf("图G2邻接表\n"); G2.DispAdjGraph();
    printf("求解结果\n");
    printf("    图G2中是否有回路: %s\n",Cycle(G2)==true?"有":"没有");
    return 0;
}
```

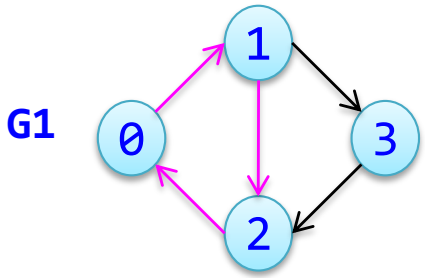
G1

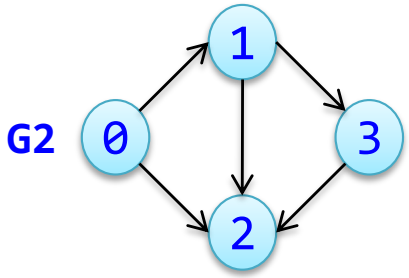


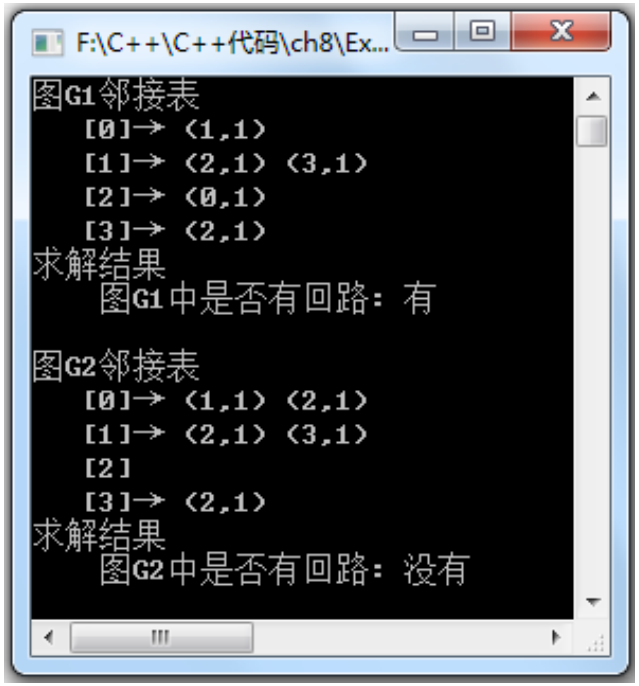
G2



程序验证


G1
1
0
3
2

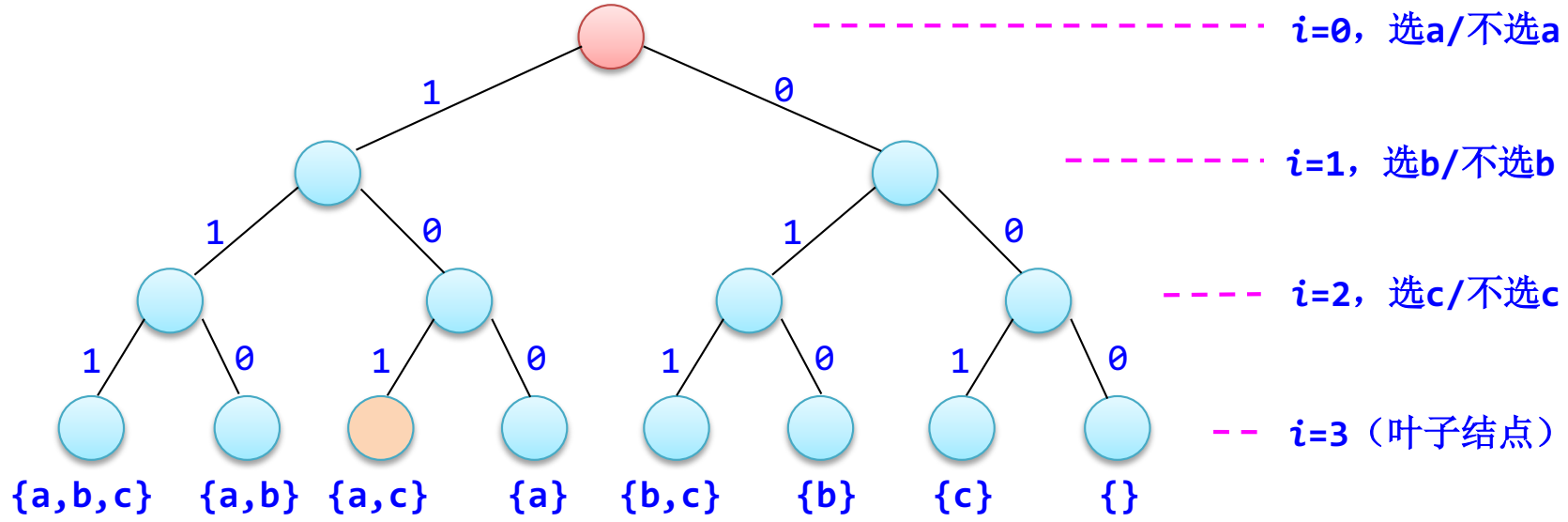

G2
1
0
3
2


F:\C++\C++代码\ch8\Ex...
图G1邻接表
[0]→ (1,1)
[1]→ (2,1) (3,1)
[2]→ (0,1)
[3]→ (2,1)
求解结果
图G1中是否有回路: 有
图G2邻接表
[0]→ (1,1) (2,1)
[1]→ (2,1) (3,1)
[2]
[3]→ (2,1)
求解结果
图G2中是否有回路: 没有

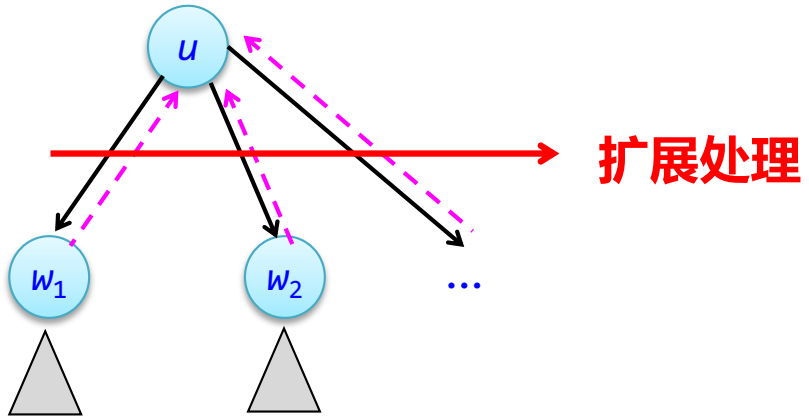
8.4.2* 回溯法

- 任何复杂的问题求解都是一步一步完成的，每一步做一个选择，最后的解可以用解向量$x=(x_0, x_1, \cdots, x_{n-1})$表示，其中分量$x_i$对应第$i$步的选择，通常可以有两个或者多个取值，表示为$x_i \in S_i$，$S_i$为$x_i$的取值候选集。
- x中各个分量x_i所有取值的组合即$S_0 \times S_1 \times \cdots \times S_{n-1}$构成问题的解向量空间，解向量空间通常用树结构表示，所以也称为解空间树。

- 例如，一个求解问题是求集合s={a，b，c}的幂集，对应的解向量为$x=(x_0, x_1, x_2)$，x_i表示元素s_i的选择和不选择两种情况（x_i=1表示选择s_i，x_i=0表示不选择s_i）。
- 树中从第0层结点到第1层结点（通常用i表示结点层次）分支上所标记的数字表示x_0可能的取值，类似地，从第i层结点到第i+1层结点分支上所标记的数字表示x_i可能的取值，从中看出，x_i只有0或者1两种取值。
- 从根结点到叶子结点路径上的标记数字构成了问题的一个可能的解，如x=(1，0，1}对应的集合为{a，c}。

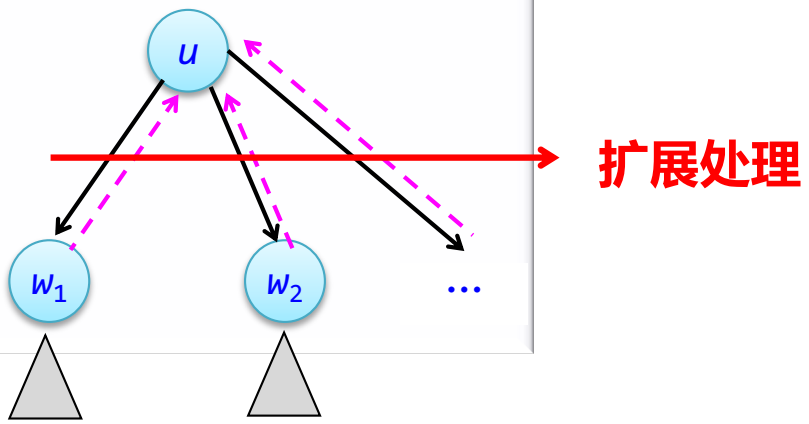


- 如何在包含问题的所有解的解空间树中求解呢？回溯法就是一种强大求解问题的通用方法。
- 基本思路：在解空间树中从根结点出发按照深度优先的顺序向下层扩展结点，扩展到某一层时，若已经无法继续扩展并且仍然未找到解，则回退到双亲结点，从双亲结点的下一个分支开始，按同样的方式继续扩展，…，直到找到问题的解或者证明无解。
- 例如在图遍历中，顶点u的扩展就是查找它的邻接点w_1，w_2，…，从顶点u扩展到顶点w_1，再从w_1按同样的方式继续扩展，若该方向没有找到解，则从w_1回退到u，再从u扩展到顶点w_2，以此类推。



回溯法基本递归框架

```
backtrack(u,i)
{   if (当前结点u满足解条件)
    {   产生一个解;
        return;
    }
    for 对当前结点u做扩展处理（假设当前结点u的子结点是wi）
    {   if (wi满足扩展要求)
        {   做u到wi的扩展操作
            backtrack(wi,i+1)
            做wi到u的回退操作
        }
    }
}
```



例如，求s="abc"（*n*=3）的幂集，采用回溯法递归框架设计的算法如下：

```
void PSet(int i,int x[])                    //输出s的幂集
{  if (i>=n)                                //到达解空间树的叶子结点
   {  disp(x);
      return;
   }
   for (int j=0;j<=1;j++)                   //扩展:x[i]的所有取值为0或者1
   {  x[i]=j;                               //s[i]取值j
      PSet(i+1,x);
   }
}
```



简化

```
void PSet(int i,int x[])                    //输出s的幂集
{  if (i>=n)                                //到达解空间树的叶子结点
      disp(x);                              //由解向量x输出s的一个子集
   else                                     //未到解空间树的叶子结点
   {  x[i]=1; PSet(i+1,x);                  //选择s[i]
      x[i]=0; PSet(i+1,x);                  //不选择s[i]
   }
}
```

【例8.8】有表达式为1□2□3□4，其中□为正或者负号，采用回溯法求出该表达式绝对值为4的所有表达式。

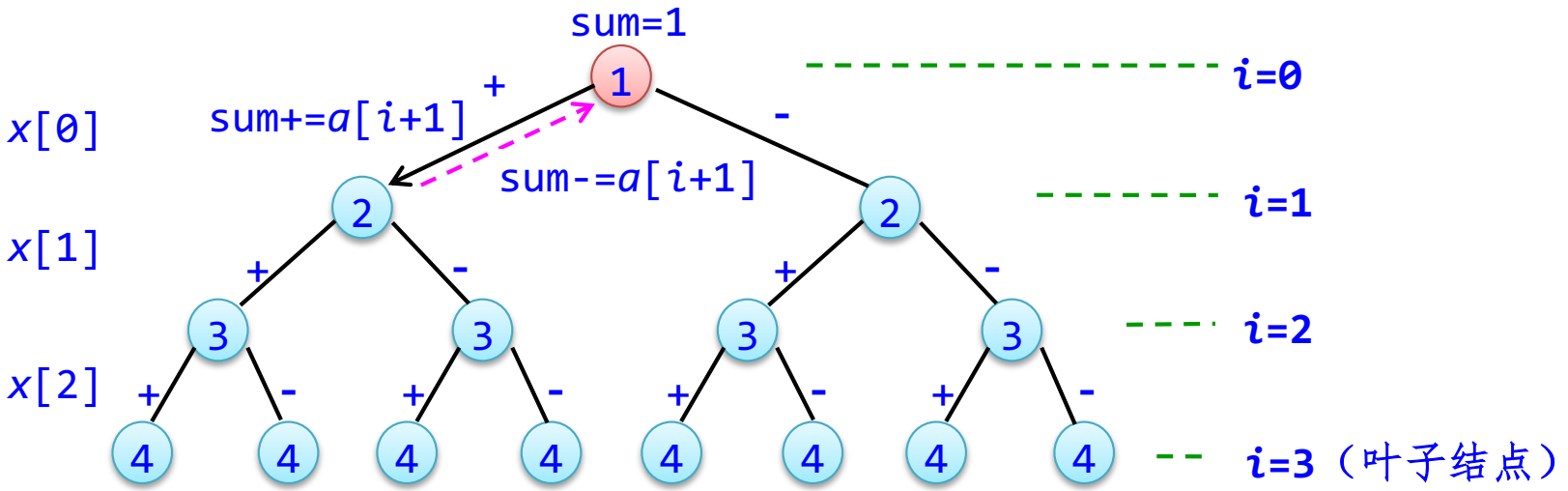


- 用数组*a*存放{1，2，3，4}（*n*=4），解向量为*x*，表达式为“1 *x*[0] 2 *x*[1] 3 *x*[2] 4”，其中*x*[*i*]取值'+'或者'-'，用sum表示当前表达式值。

例如，一个解是：1+2-3+4=4

解的表示：a={1,2,3,4}，x={+,-}

- 解空间树中i用于遍历a，也表示结点层次，根结点有$i=0$，sum=1。第i层结点左右分支表示选择运算符+或者-，选择+时的扩展操作是sum+=$a[i+1]$，对应的回退操作是sum-=$a[i+1]$。



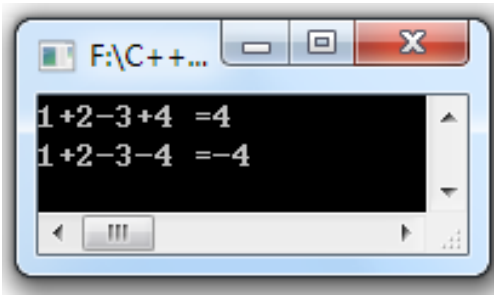
对应的程序

```
#include<iostream>
#include<cstring>
using namespace std;
#define MAXN 10
int a[]={1,2,3,4};
int n=4;
char x[MAXN];
```

```
void solve(int i,int sum)                    //回溯法求解
{  if (i==n-1)                               //到达叶子结点
   {  if (abs(sum)==4)                       //找到一个可行解
      {  printf("%d",a[0]);
         for (int j=1;j<n;j++)   //输出结果
         {   printf("%c",x[j-1]);
             printf("%d",a[j]);
          }
          printf("=%d\n",sum);
      }
   }
   x[i]='+';                                 //x[i]取'+'号
   sum+=a[i+1];                              //计算sum
   solve(i+1,sum);                           //进入下一层
   sum-=a[i+1];                              //回退恢复sum
   x[i]='-';                                 //x[i]取'-'号
   sum-=a[i+1];                              //计算sum
   solve(i+1,sum);                           //进入下一层
   sum+=a[i+1];                              //回退恢复sum
}
```

程序验证

```
int main()
{   int sum=1;                          //根结点取值1
    int i=0;                            //从根结点开始搜索解
    solve(i,sum);
    return 0;
}
```



```
void backtrack(const vector<int>& nums, int idx, int currentSum,
               const string& expr, int target, vector<string>& results) {
    if (idx == nums.size()) {
        if (abs(currentSum) == target) {
            results.push_back(expr);
        }
        return;
    }
    // 第一个数字前面不加符号
    if (idx == 0) {
        backtrack(nums, idx + 1, nums[0], to_string(nums[0]), target, results);
    } else {
        // 尝试 '+'
        backtrack(nums, idx + 1, currentSum + nums[idx],
                  expr + "+" + to_string(nums[idx]), target, results);
        // 尝试 '-'
        backtrack(nums, idx + 1, currentSum - nums[idx],
                  expr + "-" + to_string(nums[idx]), target, results);
    }
}

vector<string> solveBacktrack(const vector<int>& nums, int target) {
    vector<string> results;
    if (nums.empty()) return results;
    backtrack(nums, 0, 0, "", target, results);
    return results;
}
```

```cpp
vector<string> solveDFS(const vector<int>& nums, int target) {
    vector<string> results;
    if (nums.empty()) return results;
    // 栈中元素: (index, currentSum, expression)
    stack<tuple<int, int, string>> st;
    st.push({0, 0, ""});

    while (!st.empty()) {
        auto [idx, sum, expr] = st.top();
        st.pop();

        if (idx == nums.size()) {
            if (abs(sum) == target) {
                results.push_back(expr);
            }
            continue;
        }
        if (idx == 0) {
            // 第一个数字直接压入
            st.push({idx + 1, nums[0], to_string(nums[0])});
        } else {
            // 注意栈是后进先出，为了保持顺序一致，先压 '-' 再压 '+'
            st.push({idx + 1, sum - nums[idx], expr + "-" + to_string(nums[idx])});
            st.push({idx + 1, sum + nums[idx], expr + "+" + to_string(nums[idx])});
        }
    }
    return results;
}
```

```
------------------------------------------------------------
测试: 三个数字 1?2?3, 目标0
数字序列: [1, 2, 3], 目标绝对值: 0
回溯法找到 1 个表达式: 1+2-3
DFS法 找到 1 个表达式: 1+2-3
------------------------------------------------------------
测试: 五个数字 1?2?3?4?5, 目标5
数字序列: [1, 2, 3, 4, 5], 目标绝对值: 5
回溯法找到 3 个表达式: 1+2+3+4-5  1-2-3+4+5  1-2-3+4-5
DFS法 找到 3 个表达式: 1+2+3+4-5  1-2-3+4+5  1-2-3+4-5
------------------------------------------------------------
测试: 大数字: 10?20?30, 目标0
数字序列: [10, 20, 30], 目标绝对值: 0
回溯法找到 1 个表达式: 10+20-30
DFS法 找到 1 个表达式: 10+20-30
------------------------------------------------------------
测试: 长序列: 1?2?3?4?5?6, 目标1
数字序列: [1, 2, 3, 4, 5, 6], 目标绝对值: 1
回溯法找到 5 个表达式: 1+2+3+4-5-6  1+2+3-4+5-6  1-2+3-4-5+6  1-2-3+4+5-6  1-2-3+4-5+6
DFS法 找到 5 个表达式: 1+2+3+4-5-6  1+2+3-4+5-6  1-2+3-4-5+6  1-2-3+4+5-6  1-2-3+4-5+6
------------------------------------------------------------
```

回溯法与深度优先遍历比较

- 两者相同点都是采用深度优先顺序。
- 不同点

- 回溯法是一种解决问题的算法范式，它在搜索过程中“尝试–检查–回退”，本质上就是对解空间树的DFS，并常常结合剪枝来减少搜索量；
- 深度优先遍历目的是“遍历”，本质是无序的，也就是说访问次序不重要，重要的是都被访问过了，而回溯法是一种解决问题的算法“求解过程”，本质是有序的，也就是说必须每一步都是要求的次序。
- 深度优先遍历中已经访问过的顶点不再访问，所有顶点仅访问一次，而回溯法中已经访问过的顶点可能再次访问，也可能存在没有访问过的顶点。

可以简单理解为回溯法将问题求解转换为解空间树的深度优先遍历

【拓展】N皇后问题

在一个 N×N 的国际象棋棋盘上放置 N 个皇后，使得任意两个皇后都不能互相攻击。也就是说，任意两个皇后都不能处于同一行、同一列或同一条斜线上。

【输入格式】

一个整数 N（1 ≤ N ≤ 12），表示棋盘的大小和皇后的数量。

【输出格式】

- 第一行输出一个整数 K，表示总共有多少种不同的解法。
- 接下来 K 行，每行输出一种解法。
- 解法的表示方法：使用一个长度为 N 的整数数组，其中第 i 个元素表示第 i 行皇后所在的列号。

例如：对于 4 皇后问题，解法[2,4,1,3]表示：

第 1 行皇后在第 2 列

第 2 行皇后在第 4 列

第 3 行皇后在第 1 列

第 4 行皇后在第 3 列

标准回溯法

```
class NQueensBacktrack {
    vector<int> board;   // board[row] = col
    int N, count;

    bool isSafe(int row, int col) {
        for (int r = 0; r < row; ++r) {
            if (board[r] == col || abs(board[r] - col) == abs(r - row))
                return false;
        }
        return true;
    }
    void solve(int row) {
        if (row == N) {
            ++count;                    // 找到一个解
            return;
        }
        for (int col = 0; col < N; ++col) {
            if (isSafe(row, col)) {
                board[row] = col;
                solve(row + 1);     // board[row] 会被后续循环自然覆盖，无需显式撤销
            }
        }
    }
    . . . . . .
};
```

时间复杂度：O(N·N!)。实际由于剪枝极强，N=8 时仅访问约 2000 个节点，N=15 时约几十万，远小于 N! 。若只统计解的数量（不构造棋盘），时间复杂度与解的总数 S(N)近乎线性关系，加上剪枝检查开销。

空间复杂度：递归栈深度 O(N)

位运算回溯法

```
class NQueensBitwise {
    int N, count;
    int fullMask;                   // 低 N 位全 1

    void solve(int row, int cols, int diag1, int diag2) {
        if (row == N) {
            ++count;
            return;
        }
        // 可放的位置：列未占用，且两条对角线未占用
        int available = fullMask & ~(cols | diag1 | diag2);
        while (available) {
            int p = available & -available;    // 取最低位的 1
            available ^= p;                    // 移除该位
            solve(row + 1,
                  cols | p,
                  (diag1 | p) << 1,            // 主对角线下一行左移
                  (diag2 | p) >> 1);           // 副对角线下一行右移
        }
    }
public:
    int totalNQueens(int n) {
        N = n;
        count = 0;
        fullMask = (1 << n) - 1;
        solve(0, 0, 0, 0);
        return count;
    }
};
```

构造法

```
vector<int> constructNQueens(int n) {
    vector<int> solution(n);
    if (n % 6 != 2 && n % 6 != 3) {
        // 序列：偶数 2,4,6,... 然后奇数 1,3,5,...
        int idx = 0;
        for (int i = 2; i <= n; i += 2) solution[idx++] = i;
        for (int i = 1; i <= n; i += 2) solution[idx++] = i;
    } else {
        // n % 6 == 2 或 n % 6 == 3 的情况
        vector<int> even, odd;
        for (int i = 1; i <= n; ++i) {
            if (i % 2 == 0) even.push_back(i);
            else odd.push_back(i);
        }
        if (n % 6 == 2) {
            // 奇数部分：将 1 和 3 交换，并把 5 移到末尾
            swap(odd[0], odd[1]);           // 1 ⇄ 3
            odd.erase(odd.begin() + 2);     // 移除 5 （索引变为2）
            odd.push_back(5);               // 将 5 加到末尾
        } else { // n % 6 == 3
            // 偶数部分：把 2 移到末尾
            even.erase(even.begin());
            even.push_back(2);
            // 奇数部分：把 1 和 3 移到末尾
            odd.erase(odd.begin());
            odd.erase(odd.begin());
            odd.push_back(1);
            odd.push_back(3);
        }
        int idx = 0;
        for (int v : even) solution[idx++] = v;
        for (int v : odd) solution[idx++] = v;
    }
    // 此时 solution[i] 是第 i 行（0-based）皇后所在列（1-based值）
    // 转换为0-based列索引
    for (int &v : solution) v -= 1;
    return solution;
}
```

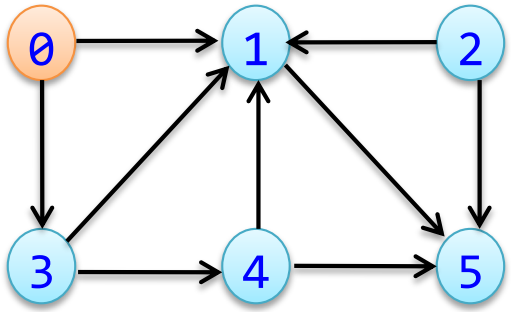
最小冲突法

```
vector<int> constructNQueens(int n) {
    vector<int> solution(n);
    if (n % 6 != 2 && n % 6 != 3) {
        // 序列：偶数 2,4,6,... 然后奇数 1,3,5,...
        int idx = 0;
        for (int i = 2; i <= n; i += 2) solution[idx++] = i;
        for (int i = 1; i <= n; i += 2) solution[idx++] = i;
    } else {
        // n % 6 == 2 或 n % 6 == 3 的情况
        vector<int> even, odd;
        for (int i = 1; i <= n; ++i) {
            if (i % 2 == 0) even.push_back(i);
            else odd.push_back(i);
        }
        if (n % 6 == 2) {
            // 奇数部分：将 1 和 3 交换，并把 5 移到末尾
            swap(odd[0], odd[1]);           // 1 ⇄ 3
            odd.erase(odd.begin() + 2);     // 移除 5 （索引变为2）
            odd.push_back(5);               // 将 5 加到末尾
        } else { // n % 6 == 3
            // 偶数部分：把 2 移到末尾
            even.erase(even.begin());
            even.push_back(2);
            // 奇数部分：把 1 和 3 移到末尾
            odd.erase(odd.begin());
            odd.erase(odd.begin());
            odd.push_back(1);
            odd.push_back(3);
        }
        int idx = 0;
        for (int v : even) solution[idx++] = v;
        for (int v : odd) solution[idx++] = v;
    }
    // 此时 solution[i] 是第 i 行（0-based）皇后所在列（1-based值）
    // 转换为0-based列索引
    for (int &v : solution) v -= 1;
    return solution;
}
```

N皇后不同解法比较

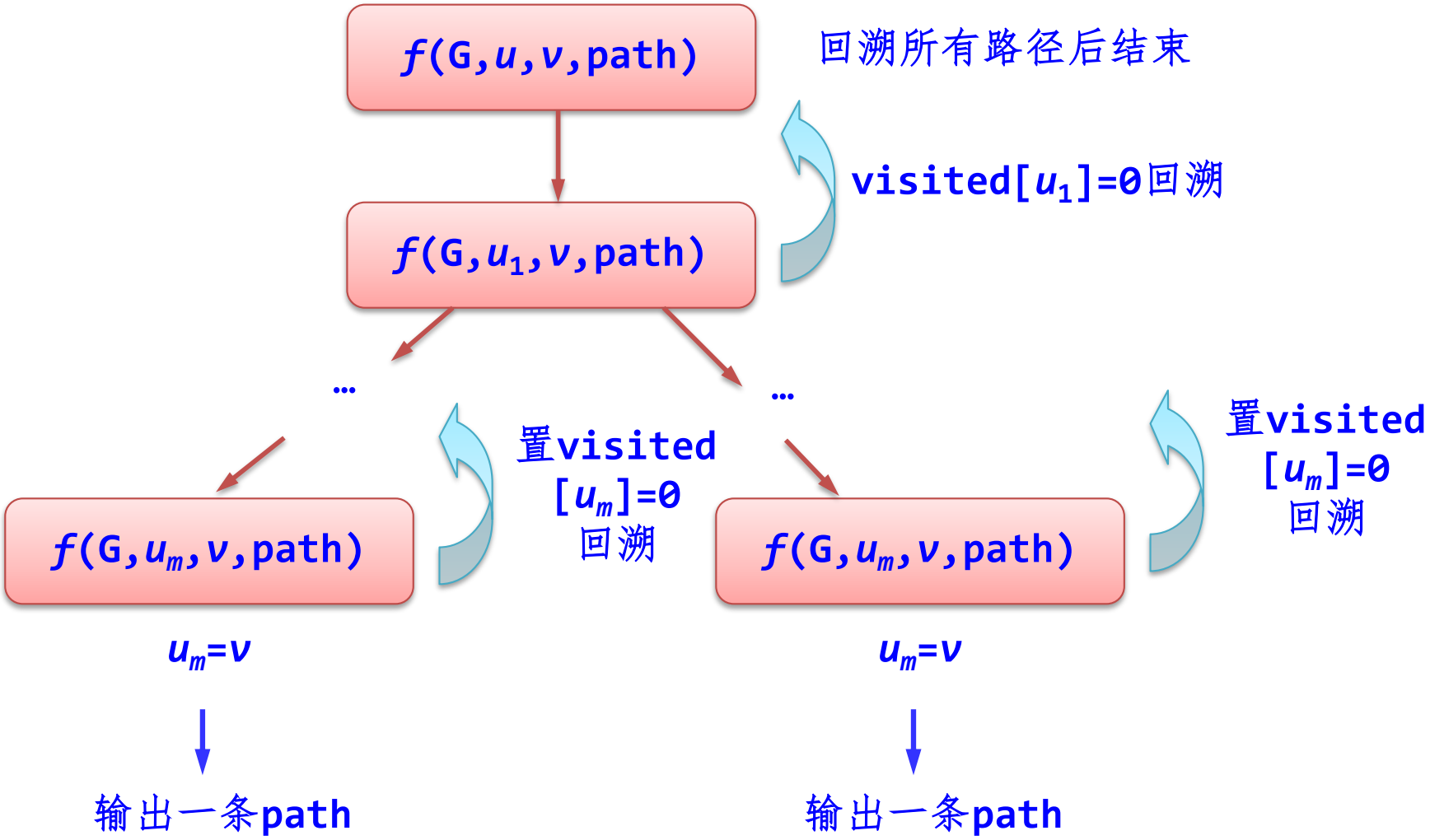
解法	目标	时间复杂度	空间复杂度	适用规模
标准回溯	求所有解	最坏 O(N·N!)，实际与解数量相关	O(N)	N ≤ 20
位运算回溯	求所有解	与解数量同阶，常数极小	O(N)	N ≤ 32/64
构造法	求一个解	O(N)	O(N)	任意 N ≥ 4
最小冲突	求一个解	平均 O(N)，最坏指数	O(N)	N 可达百万

【例8.9】假设图G采用邻接表存储，设计一个算法求顶点u到顶点v之间的所有简单路径（假设两顶点之间存在一条或多条简单路径）。并对于以下有向图，求从顶点0到顶点5的所有简单路径。



解法1

- 用path存放一条路径，每次找到终点就输出，这样得到所有的路径。
- 采用带回溯的深度优先遍历方法。

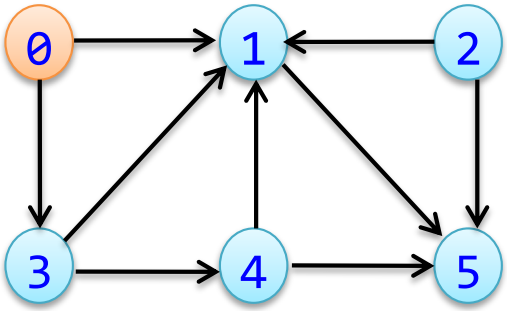

f(G,u,v,path)
回溯所有路径后结束
visited[u1]=0回溯
f(G,u1,v,path)
…
…
置visited
[um]=0
回溯
置visited
[um]=0
回溯
f(G,um,v,path)
f(G,um,v,path)
um=v
um=v
输出一条path
输出一条path

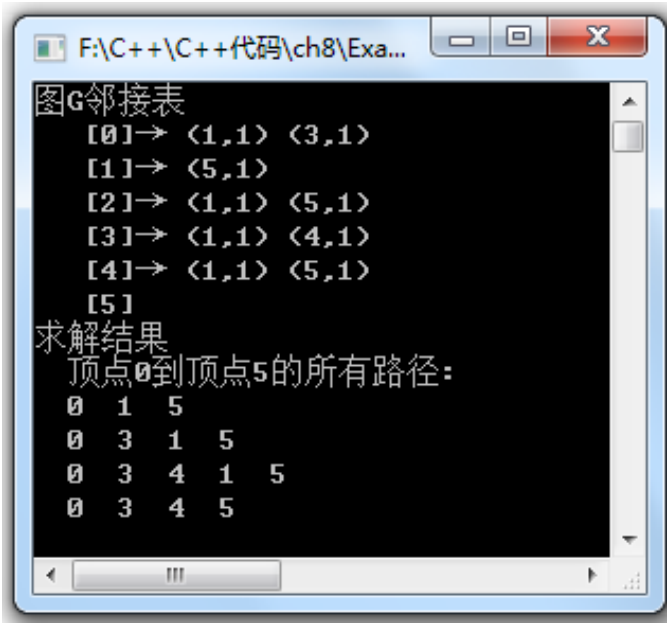
```
void FindallPath11(AdjGraph& G,int u,int v,vector<int> path)
//被FindallPath1调用
{  visited[u]=1;
   path.push_back(u);                    //顶点u加入到路径中
   if (u==v)                             //找到一条路径后输出并返回
   {  for (int i=0;i<path.size();i++)
         printf("  %d",path[i]);
      printf("\n");
      visited[u]=0;                      //回溯,重置visited[u]为0
      return;
   }
   ArcNode* p=G.adjlist[u].firstarc;
   while (p!=NULL)
   {  int w=p->adjvex;                   //找到u的邻接点w
      if (visited[w]==0)                 //若顶点w没有访问
         FindallPath11(G,w,v,path);      //从w出发继续查找
      p=p->nextarc;
   }
   visited[u]=0;                         //回溯,重置visited[u]为0
}
```

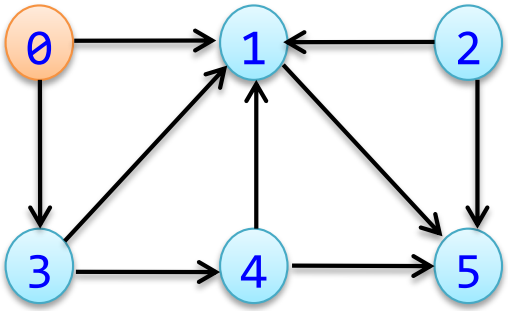
从u回退到路径上的前一个顶点

```
void FindallPath1(AdjGraph& G,int u,int v)
//解法1：求u到v的所有简单路径
{  memset(visited,0,sizeof(visited));
   vector<int> path;                    //path存放搜索路径
   FindallPath11(G,u,v,path);
}
```

程序验证

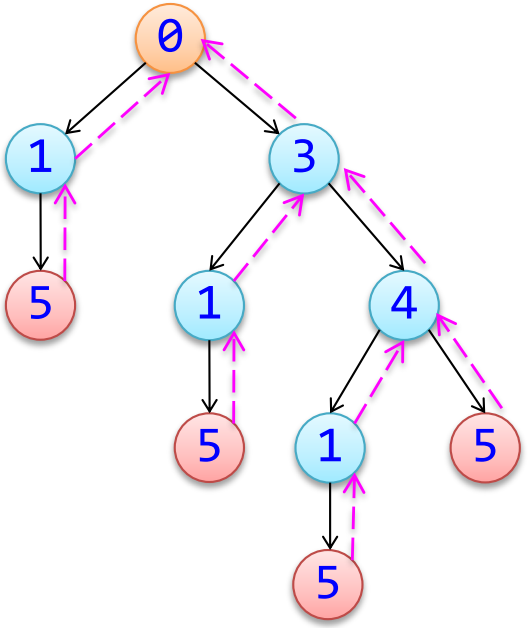






求顶点0到顶点5的所有简单路径：





① 0 → 1 → 5

② 0 → 3 → 1 → 5

③ 0 → 3 → 4 → 1 → 5

④ 0 → 3 → 4 → 5

所有路径搜索完毕

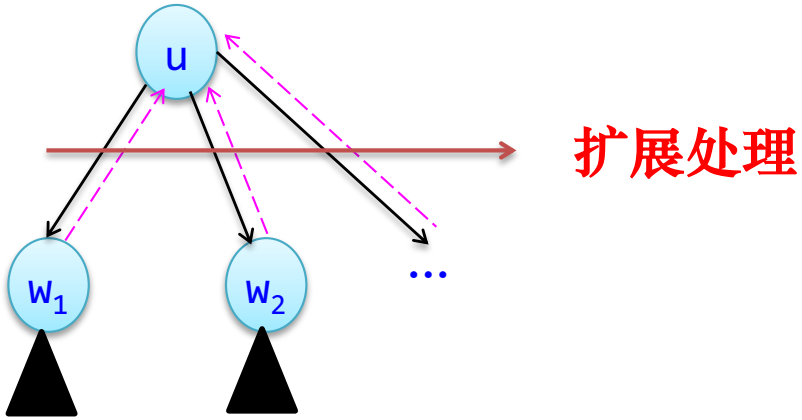
解法2 inpath数组：表示一个顶点是否在搜索路径中

回溯法 ⇨ 从顶点u出发搜索到顶点v的所有简单路径的过程如下：

① 将顶点w_i添加到path中，同时置inpath[w_i]为1。

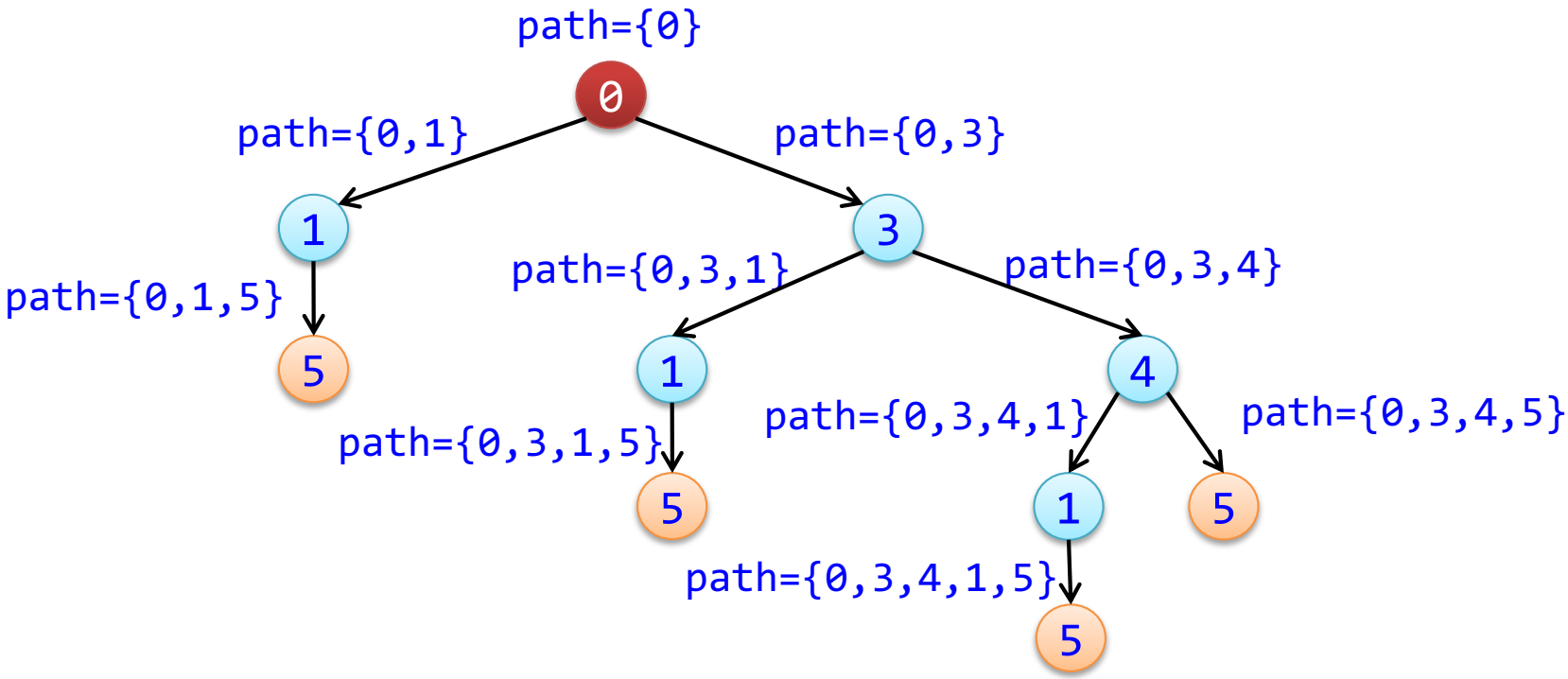
② 从顶点w_i出发搜索，其过程与从顶点u出发的搜索过程类似。

③ 从顶点w_i回退到顶点u，即将w_i从path中删除并置inpath[w_i]为0。



扩展处理

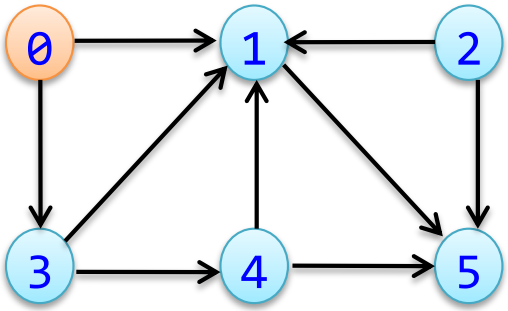
搜索0→5所有简单路径的部分解空间树

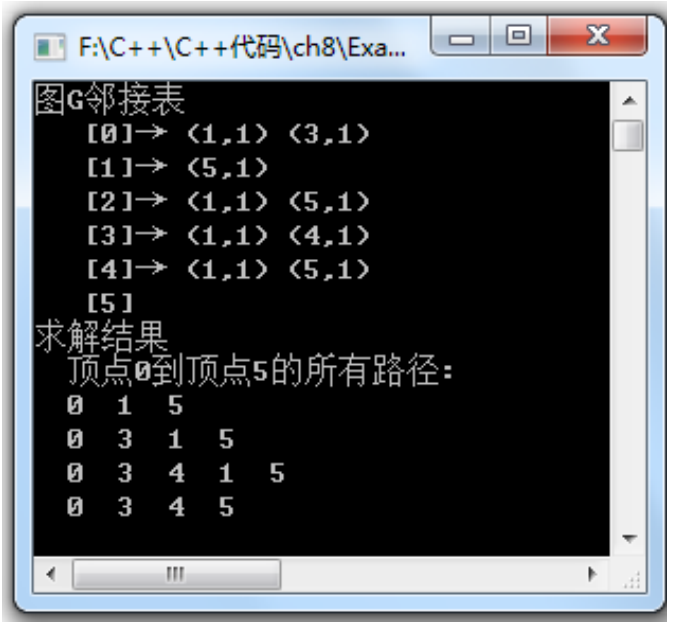


```
int inpath[MAXV];                                   //全局数组
vector<int> path;                                   //全局变量
void FindallPath21(AdjGraph& G,int u,int v)
//被Findapath2函数调用
{  if (u==v)                                        //叶子结点：找到一条路径后输出
   {  for (int i=0;i<path.size();i++)               //输出一条路径
         printf("  %d",path[i]);
      printf("\n");
      return;
   }
   ArcNode* p=G.adjlist[u].firstarc;                //扩展u
   while (p!=NULL)
   {  int w=p->adjvex;                              //找到u的邻接点w
      if (inpath[w]==0)                             //若顶点w不在path中
      {  path.push_back(w);                         //将顶点w添加的path中
         inpath[w]=1;                               //置w已经在path中
         FindallPath21(G,w,v);                      //递归调用
         path.pop_back();                           //path回退
         inpath[w]=0;                               //置w不在path中
      }
      p=p->nextarc;
   }
}
```

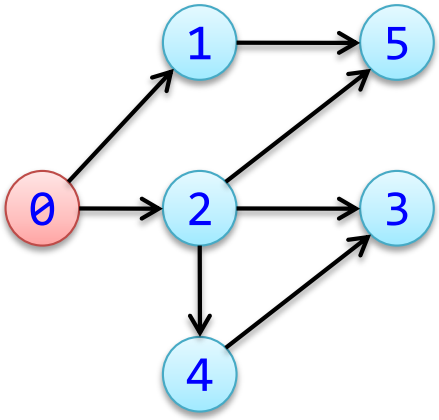
```
void FindallPath2(AdjGraph& G,int u,int v)
//解法2:求u到v的所有简单路径
{  memset(inpath,0,sizeof(inpath));          //初始化inpath
   path.push_back(u);                        //将顶点u添加的path中
   inpath[u]=1;                              //置u已经在path中
   FindallPath21(G,u,v);
}
```

程序验证



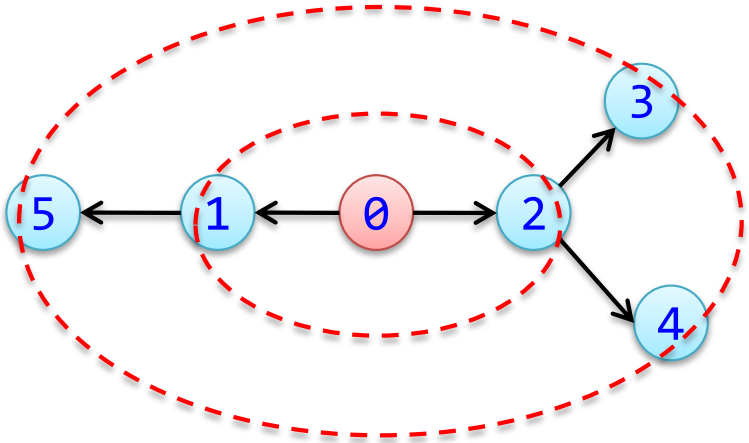


8.4.3 广度优先遍历算法的应用



起始点0到图中其他顶点的最短路径长度：

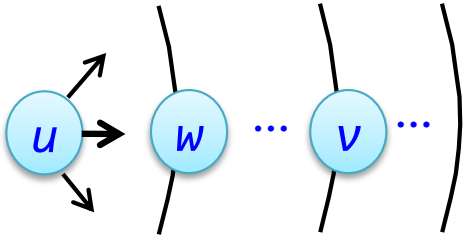
0 : 0
0 → 1 : 1
0 → 2 : 1
0 → 1 → 5 : 2
0 → 2 → 3 : 2
0 → 2 → 4 : 2



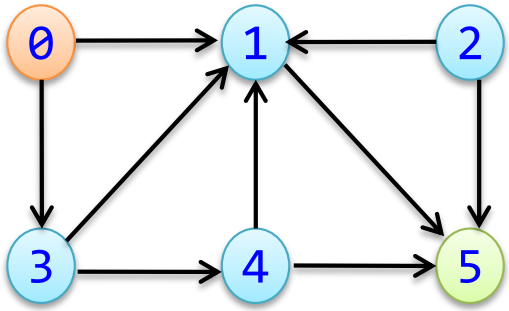
【例8.10】假设图G采用邻接表存储，设计一个算法，求不带权图G中从顶点u到顶点v的一条最短路径长度。



- 图G是不带权图，一条边的长度计为1，因此求顶点u到顶点v的最短路径就是求顶点u到v的边数最少的顶点序列，其长度就是最短路径长度。
- 利用广度优先遍历算法，从u出发进行广度优先遍历，类似于从顶点u出发一层一层地向外伸展，当第一次找到顶点v时队列中便包含了从顶点u到顶点v最短路径。
- 实际上不必在找到顶点v后再求最短路径长度，而是在求从顶点u到顶点v的伸展次数即可。

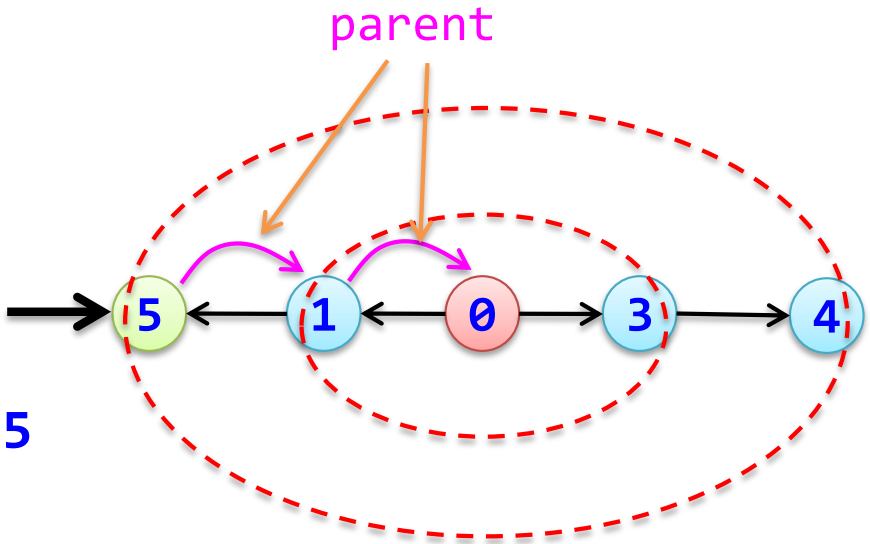


按距离u的最短路径长度一层一层地访问其他顶点



起始点0到图中其他顶点的最短路径长度：

0 : 0
0 → 1 : 1
0 → 3 : 1
0 → 1 → 5 : 2
0 → 3 → 4 : 2



5 1 0
反向：0 → 1 → 5

```
struct QNode                        //队列元素类型
{   int v;                          //顶点编号
    int dis;                        //源点到当前顶点的最短路径长度

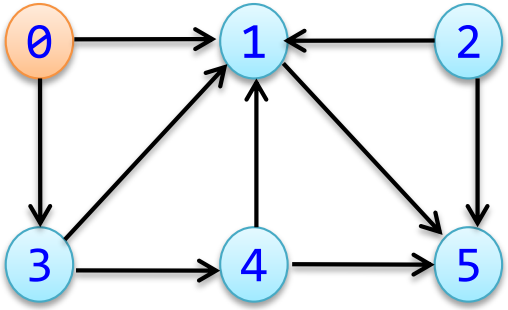
    QNode() {}                      //构造函数
    QNode(int v,int dis)            //重载构造函数
    {   this->v=v;
        this->dis=dis;
    }
};
```

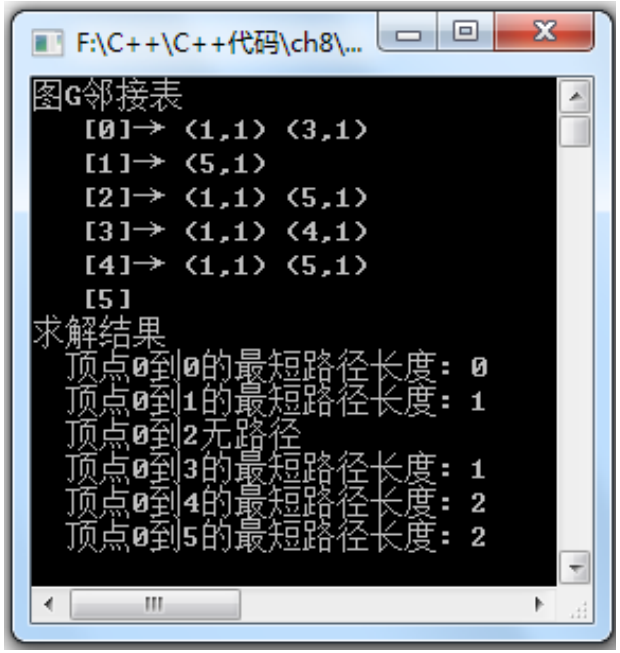
```
int Shortdist(AdjGraph& G,int u,int v) //求u到v的最短路径长度
{  int visited[MAXV];                   //访问标记数组
   memset(visited,0,sizeof(visited));
   queue<QNode> qu;                     //定义一个队列qu
   visited[u]=1;                        //置已访问标记
   qu.push(QNode(u,0));                 //起始点u(距离为0)进队
   while (!qu.empty())                  //队不空循环
   {  QNode e=qu.front(); qu.pop();     //出队一个元素e
      if (e.v==v)                       //找到顶点v
         return e.dis;                  //返回u到v的最短路径长度

      ArcNode* p=G.adjlist[e.v].firstarc;
      while (p!=NULL)
      {  int w=p->adjvex;               //找到u的邻接点w
         if (visited[w]==0)             //若顶点w没有访问
         {  visited[u]=1;               //置已访问标记
            qu.push(QNode(w,e.dis+1)); //邻接点w进队
         }
         p=p->nextarc;
      }
   }
   return INF;                          //没有路径的情况
}
```

程序验证

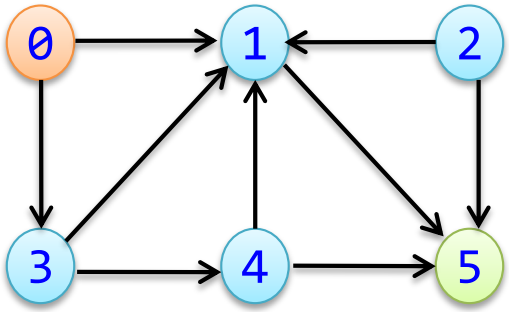
```
int main()
{  AdjGraph G;
   int n=6,e=9;
   int a[MAXV][MAXV]={{0,1,0,1,0,0},{0,0,0,0,0,1},{0,1,0,0,0,1},
                      {0,1,0,0,1,0},{0,1,0,0,0,1},{0,0,0,0,0,0}};
   G.CreateAdjGraph(a,n,e);
   printf("图G邻接表\n"); G.DispAdjGraph();
   printf("求解结果\n");
   int u=0;
   for (int v=0;v<n;v++)
   {  int spathlen=Shortdist(G,u,v);
      if (spathlen==INF)
         printf("  顶点%d到%d无路径\n",u,v);
      else
         printf("  顶点%d到%d的最短路径长度: %d\n",u,v,spathlen);
   }
   return 0;
}
```

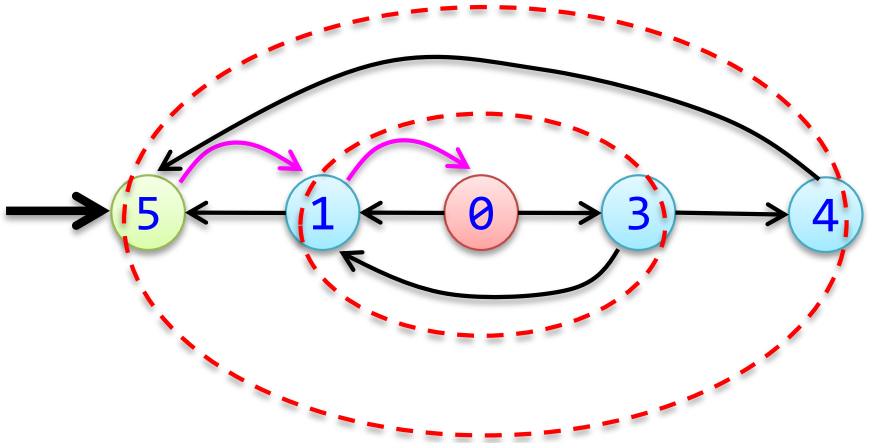

0
1
2
3
4
5


F:\C++\C++代码\ch8\...
图G邻接表
[0]→ (1,1) (3,1)
[1]→ (5,1)
[2]→ (1,1) (5,1)
[3]→ (1,1) (4,1)
[4]→ (1,1) (5,1)
[5]
求解结果
顶点0到0的最短路径长度: 0
顶点0到1的最短路径长度: 1
顶点0到2无路径
顶点0到3的最短路径长度: 1
顶点0到4的最短路径长度: 2
顶点0到5的最短路径长度: 2



为什么广度优先遍历找到的路径一定是最短路径呢?





- 广度优先遍历找到的路径上的每个顶点均为不同层次的顶点，所以该路径一定是最短路径。
- 深度优先遍历找到的路径上的顶点可能属于相同层次的顶点，所以不一定是最短路径。